1 Introdução

[TODO]

1.1 Objetivos

[TODO]

1.1.1 Objetivo Geral

[TODO]

1.1.2 Objetivos Específicos

[TODO]

1.2 Organização da monografia

[TODO]

Este trabalho é composto por XX capítulos. Neste Capítulo 1, proporcionamos uma introdução, justificativa e delineamos os objetivos...

2 Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para a compreensão do trabalho. Inicialmente, são introduzidos fundamentos de aprendizado profundo e redes neurais artificiais, seguidos pelo processo de treinamento dessas redes. Em seguida, discute-se o problema do consumo energético em modelos de *Deep Learning* e, por fim, são apresentadas as técnicas de compressão de modelos, com destaque para a poda neural (*pruning*), que é o foco deste trabalho.

A exposição é deliberadamente progressiva e não pressupõe familiaridade prévia com aprendizado de máquina. Cada conceito é introduzido primeiro por meio de uma analogia com situações cotidianas e só então formalizado, de modo que o leitor possa acompanhar o raciocínio antes de encontrar a notação matemática que o descreve.

2.1 Aprendizado Profundo e Redes Neurais

2.1.1 Aprendizado Profundo (Deep Learning)

O Aprendizado Profundo é um campo especializado de aprendizado de máquina (GOODFELLOW; BENGIO; COURVILLE, 2016; LECUN; BENGIO; HINTON, 2015), caracterizado pelo uso de redes neurais artificiais com múltiplas camadas, o que permite aprender e extrair informações complexas a partir de grandes volumes de dados.

2.1.2 Redes Neurais Artificiais

A rede neural artificial é um modelo computacional inspirado na estrutura do cérebro humano, que busca imitar a forma como neurônios se conectam e processam sinais para aprender a partir de dados, identificando padrões e tomando decisões. Um exemplo clássico é o perceptron multicamadas (MLP), um modelo composto por funções matemáticas que mapeiam valores de entrada em valores de saída (GOODFELLOW; BENGIO; COURVILLE, 2016). Em tarefas de visão computacional, por exemplo, milhões de imagens de um objeto podem ser processadas para que a rede aprenda suas características visuais e seja capaz de reconhecê-lo posteriormente.

Uma forma de tentar enteder a definição de uma RNN é, imagine uma comissão encarregada de decidir se um pedido de empréstimo deve ser aprovado. Nenhum membro decide sozinho, cada um examina um aspecto diferentee, como a renda declarada, o histórico de pagamentos, o tempo de emprego e então, após analisar todos os aspectos, emite um parecer. Esses pareceres não têm todos o mesmo valor, pois a comissão confia mais

em uns avaliadores do que em outros, e a decisão final combina todos eles, ponderados por essa confiança. Em uma rede neural, cada nó é um desses avaliadores e o peso de cada conexão é exatamente a confiança depositada naquele parecer. A diferença essencial é que a rede não recebe essas confianças prontas, ela as descobre sozinha, por tentativa e erro, ao longo do treinamento.

Uma rede neural profunda é composta por unidades (nós) interconectadas, organizadas em camadas (GOODFELLOW; BENGIO; COURVILLE, 2016; NIELSEN, 2015).

- **Nó (neurônio):** É uma unidade computacional que calcula uma soma ponderada das entradas mais um *bias* e passa o resultado através de uma função de ativação, produzindo uma saída.
- **Camada de Entrada (Input Layer):** É onde o processo é iniciado e os dados são introduzidos na rede. O número de nós nessa camada corresponde ao número de características dos dados de entrada.
- **Camadas Ocultas (Hidden Layers):** São camadas intermediárias entre a entrada e a saída. Cada nó de uma camada oculta está conectado a todos os nós das camadas anterior e seguinte. É nelas que ocorre a maior parte do processamento e do aprendizado; a saída de uma camada oculta é utilizada como entrada da próxima, e, a cada camada, a rede passa a detectar características mais complexas e abstratas. Redes com múltiplas camadas ocultas são as chamadas redes de *Deep Learning*.
- **Camada de Saída (Output Layer):** Camada final da rede, onde o resultado é produzido. A estrutura da camada de saída depende da tarefa: em regressão, há um único nó com função de ativação linear; em classificação binária, um único nó expressa resultados do tipo sim/não; em classificação multiclasse, há um nó para cada classe possível, e o de maior ativação corresponde à predição final.

Cada conexão entre nós possui um peso associado, que determina a sua importância relativa. Pesos altos amplificam o sinal de entrada, enquanto pesos próximos de zero tornam a conexão pouco influente. Os pesos são inicializados com valores aleatórios e ajustados durante o treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.3 Bias em Redes Neurais

O *bias* é um parâmetro adicional que ajusta a saída de um neurônio. Trata-se de um valor numérico associado a cada neurônio, não ligado a nenhuma entrada específica, atuando como um coeficiente linear. Antes da aplicação da função de ativação, o *bias* é somado ao resultado da soma ponderada das entradas.

A principal função do *bias* é deslocar a função de ativação, permitindo que o modelo se ajuste melhor aos dados. Sem ele, a soma ponderada precisaria ser alta para que o neurônio fosse ativado; com o *bias*, um neurônio pode ser ativado mesmo com uma soma ponderada baixa, caso seu *bias* seja suficientemente alto (NIELSEN, 2015). Inversamente, um *bias* muito negativo dificulta a ativação do neurônio. O *bias* é um parâmetro aprendível: inicialmente é inicializado com zero ou valores próximos de zero e, durante o treinamento, é atualizado pelo mesmo processo de retropropagação e descida de gradiente utilizado para os pesos.

Retomando com a analogia da comissão de crédito, se os pesos representam o quanto se confia em cada avaliador, o *bias* representa o quanto o avaliador é exigente. Um membro gentil aprova o pedido mesmo diante de evidências modestas; um membro rigoroso só aprova quando as evidências são fortes. São duas coisas independentes — confiar muito em alguém não diz nada sobre o quanto essa pessoa é exigente — e é por isso que a rede precisa aprender os dois conjuntos de valores separadamente.

2.1.4 Função de Ativação

A função de ativação é uma função matemática aplicada ao resultado da soma ponderada das entradas de um neurônio (já acrescido do *bias*), determinando a saída do neurônio. Sua principal função é introduzir não linearidade no modelo: sem ela, a composição de diversas camadas resultaria em uma transformação puramente linear, equivalente a uma rede com uma única camada, o que limitaria drasticamente a capacidade expressiva do modelo (GOODFELLOW; BENGIO; COURVILLE, 2016).

A razão de existir dessa função merece um exemplo concreto, pois é ela que justifica o próprio empilhamento de camadas. Considere um produto de R$ 100,00 submetido a dois descontos sucessivos, um de 10% e outro de 20%: o primeiro leva o preço a R$ 90,00 e o segundo a R$ 72,00, exatamente o mesmo resultado que um único desconto de 28% produziria. Os dois passos colapsaram em um só, e o segundo nada acrescentou que um único desconto bem escolhido não fizesse sozinho. Encadear operações lineares nunca gera nada além de outra operação linear, e com redes neurais acontece o mesmo: sem função de ativação, uma rede de cem camadas calcularia exatamente aquilo que uma única camada já calcularia, e toda a profundidade seria desperdiçada.

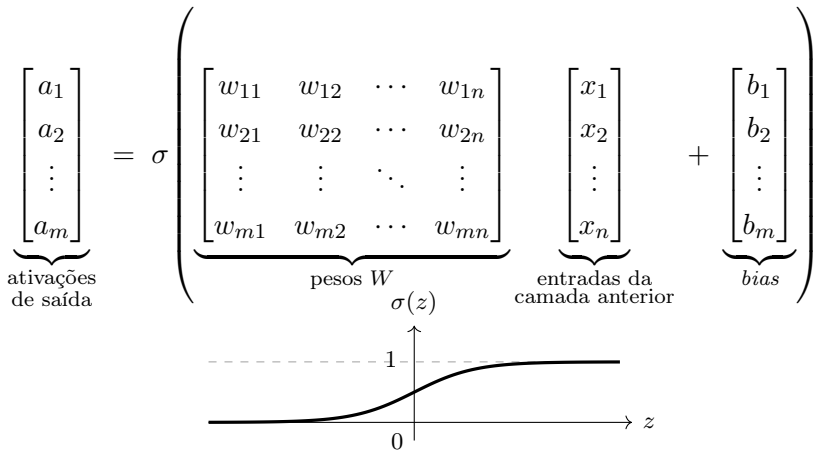
Suponha agora que a loja adote uma regra adicional, aplicada logo depois de cada desconto: nenhum produto pode ser vendido por menos de R$ 85,00. Essa regra é o análogo da função de ativação — ela não faz parte do desconto, mas incide sobre o resultado dele, antes que esse resultado siga para a etapa seguinte. Com ela, o encadeamento deixa de colapsar. O produto de R$ 100,00 cai para R$ 90,00 e depois para R$ 72,00, mas a regra o devolve a R$ 85,00, de modo que o desconto efetivo foi de 15%. Já um produto de R$ 200,00 percorre os mesmos dois descontos, chega a R$ 144,00 sem nunca esbarrar

no limite e tem desconto efetivo de 28%. A mesma sequência de operações passou a produzir resultados diferentes conforme a entrada, e nenhum desconto único reproduz esse comportamento. É esse rompimento do encadeamento que permite a cada camada acrescentar algo que a anterior não conseguia representar. A regra de preço mínimo, aliás, tem a mesma forma da ReLU, apresentada adiante nesta seção: ambas preservam os valores acima de um limiar e substituem os demais por um valor fixo.

A ativação de um neurônio ocorre em três etapas (NIELSEN, 2015):

- Soma ponderada: o neurônio multiplica o valor de ativação de cada nó da camada anterior pelo peso da respectiva conexão e soma todos os resultados.
- Adição do *bias*: ao resultado da soma ponderada é somado o valor do *bias*, que auxilia o neurônio a ativar-se mais ou menos facilidade.
- Função de ativação: o resultado final (soma ponderada acrescida do *bias*) é passado pela função de ativação, cuja saída é então transmitida aos nós da próxima camada.

Figura 1 – Representação matricial da propagação de um neurônio, da soma ponderada à função de ativação (Sigmoide).



a função de ativação comprime qualquer valor de z no intervalo $(0, 1)$

Fonte: Adaptado de Nielsen (2015).

Há diversas funções de ativação, cada uma com suas próprias características e aplicações.

- **ReLU (Rectified Linear Unit):** Função de ativação mais utilizada em redes neurais modernas, proposta para redes profundas por Nair e Hinton (2010) e popularizada por Krizhevsky, Sutskever e Hinton (2012). É uma função linear por partes, com inclinação 1 para valores positivos e 0 para valores negativos. É computacionalmente muito eficiente e não sofre do problema do desaparecimento do gradiente

para entradas positivas. Em contrapartida, pode sofrer do problema conhecido como *Dying ReLU*, no qual neurônios cujas entradas se tornam persistentemente negativas deixam de ser ativados e param de aprender.

- **Sigmoide:** Função que mapeia qualquer valor real para o intervalo $(0, 1)$, sendo útil em classificação binária, já que a saída pode ser interpretada como uma probabilidade. Caiu em desuso em camadas ocultas porque, para entradas com magnitude muito alta ou muito baixa, sua derivada se aproxima de zero, fenômeno conhecido como desaparecimento do gradiente (BENGIO; SIMARD; FRASCONI, 1994).

- **Tanh (Tangente Hiperbólica):** Semelhante à Sigmoide, porém mapeando os valores para o intervalo $(-1, 1)$, o que a torna centrada em zero e costuma favorecer a convergência. Também sofre do problema do desaparecimento do gradiente.

- **Softmax:** Diferentemente das demais, não é aplicada a neurônios individuais de camadas ocultas, mas sim ao conjunto dos neurônios da camada de saída em tarefas de classificação multiclasse. Transforma as saídas brutas da última camada em uma distribuição de probabilidades, com valores no intervalo $(0, 1)$ que somam 1. Na prática, converte pontuações sem escala definida em porcentagens comparáveis: o modelo deixa de responder "7,2 para gato e 3,1 para cachorro" e passa a responder "85% de chance de ser gato, 15% de ser cachorro".

Em cada camada, as ativações produzidas determinam as ativações da camada seguinte. De camada em camada, esses padrões tornam-se progressivamente mais complexos e abstratos, até que a camada de saída produz o resultado final. Assim, camadas iniciais aprendem características simples, enquanto camadas mais profundas combinam essas características para representar conceitos de alto nível (LECUN; BENGIO; HINTON, 2015).

2.2 Treinamento de Redes Neurais

2.2.1 Gradiente

No contexto de redes neurais, o gradiente corresponde ao vetor de derivadas parciais da função de perda em relação aos pesos da rede. Ele é o elemento central que permite a minimização da função de perda por meio do algoritmo da descida do gradiente, um método de primeira ordem que utiliza as informações do gradiente para atualizar os parâmetros da rede (GOODFELLOW; BENGIO; COURVILLE, 2016).

Formalmente, sendo L a função de perda e $\mathbf{w} = (w_1, w_2, \ldots, w_n)$ o conjunto dos n parâmetros da rede, o gradiente é o vetor que reúne a derivada parcial da perda em relação a cada um deles (GOODFELLOW; BENGIO; COURVILLE, 2016):

$$\nabla L(\mathbf{w}) = \left[\frac{\partial L}{\partial w_1},\ \frac{\partial L}{\partial w_2},\ \ldots,\ \frac{\partial L}{\partial w_n}\right] \tag{2.1}$$

Cada componente do vetor de gradiente é uma derivada parcial, calculada assumindo que as demais variáveis são mantidas constantes. A magnitude da derivada indica o quanto a função está variando; o seu sinal, a direção dessa variação: positivo, a função está aumentando; negativo, diminuindo.

Tomado como um todo, o vetor da Equação 2.1 aponta na direção em que a perda cresce mais rapidamente. É por isso que a descida do gradiente caminha no sentido oposto, $-\nabla L(\mathbf{w})$: seguir o gradiente invertido é o que aproxima a rede de um mínimo da função de perda.

Uma analogia comum é a de um caminhante que desce uma encosta em meio à neblina, enxergando apenas o solo ao seu redor: a cada passo, ele sente para qual lado o terreno mais desce (o oposto do gradiente) e avança nessa direção. Na descida do gradiente, a "encosta" é a função de perda e os "passos" são os ajustes nos pesos, repetidos até alcançar um ponto de mínimo.

2.2.2 Backpropagation

A retropropagação (*backpropagation*) é o algoritmo que permite o aprendizado das redes neurais, por possibilitar o cálculo da contribuição de cada parâmetro (peso ou *bias*) para o erro da predição (RUMELHART; HINTON; WILLIAMS, 1986). Uma vez que o erro é calculado na camada de saída, o algoritmo executa um fluxo invertido: da saída em direção à entrada, propagando o erro para as camadas anteriores.

A lógica é a mesma de uma apuração de responsabilidades em uma linha de produção. Sabe-se apenas que o produto final saiu defeituoso; para corrigir o processo, é preciso percorrer a linha de trás para frente e estimar o quanto cada etapa contribuiu para o defeito. Uma etapa que respondeu por boa parte do problema precisa de um ajuste grande; outra que quase não influiu, de um ajuste pequeno. O *backpropagation* faz esse rastreamento de forma matemática, atribuindo a cada peso da rede uma parcela de responsabilidade pelo erro.

Para saber como cada peso individual se correlaciona com o erro, o algoritmo calcula a derivada parcial da função de perda em relação a esse peso, utilizando a regra da cadeia multivariada para decompor o cálculo em derivadas intermediárias. Com todas as derivadas obtidas, um algoritmo de otimização atualiza cada peso na direção oposta ao gradiente:

$$w_{novo} = w_{antigo} - \eta \cdot \frac{\partial L}{\partial w}$$

em que η é a taxa de aprendizado, hiperparâmetro que controla o tamanho do ajuste aplicado a cada iteração.

Voltando ao caminhante na neblina, a taxa de aprendizado é o tamanho do passo que ele dá. Passos curtos demais tornam a descida segura, porém lenta a ponto de o treinamento não terminar em tempo viável, já passos longos demais fazem o caminhante descer mais rápido, mas aumenta o risco de se machucar. Encontrar um valor adequado é uma das decisões mais sensíveis do treinamento.

2.2.3 Hiperparâmetros

Hiperparâmetros são variáveis externas que definem tanto o algoritmo de aprendizado quanto a arquitetura da rede neural. Diferentemente dos parâmetros do modelo (pesos e *biases*), que são ajustados automaticamente pelo *backpropagation*, os hiperparâmetros são definidos antes do início do treinamento e controlam como a rede aprende a partir dos dados. Existem técnicas automatizadas para sua busca, como *grid search* (BERGSTRA; BENGIO, 2012) e otimização bayesiana (SNOEK; LAROCHELLE; ADAMS, 2012), mas o ajuste manual baseado em experiência ainda é amplamente empregado.

A distinção entre parâmetros e hiperparâmetros é a mesma que separa, em uma cozinha, as escolhas do cozinheiro do resultado do cozimento. A temperatura do forno e o tempo de preparo são definidos antes de o prato ir ao fogo e não se alteram sozinhos: são os hiperparâmetros. O ponto da massa e a cor da crosta, por outro lado, são produzidos pelo processo: são os parâmetros. Errar a temperatura não impede o prato de assar, mas compromete o resultado e, do mesmo modo, hiperparâmetros mal escolhidos não impedem o treinamento de ocorrer, apenas fazem com que ele chegue a um modelo pior.

Em geral, os hiperparâmetros podem ser agrupados em duas categorias:

- **Hiperparâmetros de arquitetura:** definem a estrutura da rede, como o número de camadas ocultas, o número de neurônios em cada camada e o tipo de função de ativação utilizada.
- **Hiperparâmetros de treinamento:** controlam como o modelo se ajusta aos dados, como a taxa de aprendizado, o número de épocas, o tamanho do lote (*batch size*) e a escolha do algoritmo de otimização.

2.2.4 Passo a passo do treinamento

O treinamento de uma rede neural supervisionada requer uma grande base de dados rotulada, isto é, cada exemplo de entrada deve estar associado a uma saída correta. Essa base de dados é tradicionalmente dividida em três subconjuntos (GOODFELLOW; BENGIO; COURVILLE, 2016):

- **Conjunto de treinamento**, usado para ajustar os pesos e *biases*;

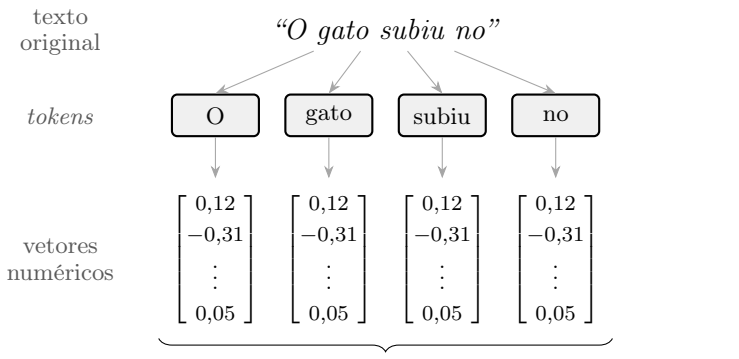
- **Conjunto de validação**, usado para monitorar o desempenho durante o treinamento e ajustar hiperparâmetros;
- **Conjunto de teste**, usado para avaliar o desempenho final do modelo de forma imparcial.

Além disso, é necessário definir a arquitetura da rede (camadas, neurônios, funções de ativação) e os hiperparâmetros de treinamento. O treinamento ocorre ao longo de diversas iterações, nas quais cada passagem completa pelo conjunto de treinamento é chamada de época (*epoch*); em cada época, os dados são processados em pequenos lotes (*batches*) (GOODFELLOW; BENGIO; COURVILLE, 2016).

A analogia mais direta é a de um estudante diante de um livro. Uma época equivale a percorrer o livro inteiro uma vez; os lotes são os trechos em que essa leitura é dividida, com uma pausa para revisão ao fim de cada um. E, assim como raramente basta uma única leitura para fixar o conteúdo, a rede precisa percorrer o conjunto de treinamento diversas vezes até que os ajustes se estabilizem.

Como exemplo, considere um modelo de linguagem que recebe a frase "O gato subiu no". Antes de qualquer cálculo, o texto precisa virar número: ele é fragmentado em unidades menores, chamadas *tokens*, e cada *token* é convertido em um vetor de valores numéricos. São esses vetores que alimentam a camada de entrada — cada posição do vetor corresponde a um nó (JURAFSKY; MARTIN, 2025).

Figura 2 – Da frase aos nós da camada de entrada: o texto é fragmentado em *tokens* e cada *token* é convertido em um vetor numérico.



Fonte: .

O ciclo de treinamento pode ser resumido nos seguintes passos (GOODFELLOW; BENGIO; COURVILLE, 2016):

- **Passo 1 – Inicialização dos parâmetros:** pesos são inicializados com valores aleatórios baixos e *biases* com zero. Essa quebra de simetria permite que diferentes neurônios aprendam características distintas.

- **Passo 2 – *Forward pass*:** um lote de dados é inserido na camada de entrada e propagado através das camadas ocultas até a camada de saída, produzindo uma predição.
- **Passo 3 – Cálculo da perda:** a predição é comparada com o rótulo verdadeiro por meio de uma função de perda, que quantifica numericamente o quão distante a predição está do valor correto.
- **Passo 4 – *Backpropagation*:** o erro é propagado de volta pela rede, calculando-se o gradiente da função de perda em relação a cada peso e *bias*.
- **Passo 5 – Atualização dos parâmetros:** um otimizador atualiza pesos e *biases* na direção oposta ao gradiente, reduzindo a função de perda.
- **Passo 6 – Repetição:** os passos 2 a 5 são repetidos para cada lote e ao longo de um número definido de épocas.

O treinamento é interrompido quando o modelo atinge uma precisão satisfatória ou quando a função de perda deixa de diminuir significativamente.

2.3 Arquitetura Transformer

A arquitetura *Transformer*, proposta por Vaswani et al. (2017) no trabalho *Attention is All You Need*, marcou uma mudança profunda na forma como modelos de aprendizado profundo processam sequências de texto. Antes dessa proposta, o campo era dominado por arquiteturas que processavam a sequência elemento por elemento, palavra por palavra. O *Transformer* rompeu com essa abordagem ao introduzir um mecanismo capaz de considerar toda a sequência de uma só vez. Esta seção apresenta, de forma progressiva, os conceitos necessários para compreender a arquitetura utilizada neste trabalho: as limitações das abordagens anteriores, o mecanismo de atenção, a operação de autoatenção e sua versão multi-cabeça, a estrutura de cada bloco da rede e, finalmente, o modelo GPT-2, que é o objeto central dos experimentos de poda.

2.3.1 O Problema que o Transformer Resolve

Toda arquitetura nasce como resposta a uma limitação da anterior, e o *Transformer* não é exceção. Compreender qual era essa limitação é o que transforma o mecanismo de atenção em uma solução, e não em uma escolha arbitrária.

Até 2017, o processamento de texto era dominado pelas redes neurais recorrentes (*Recurrent Neural Networks*, RNNs). Elas leem a frase palavra por palavra, na ordem em que aparece, carregando consigo um resumo do que já foi lido: a cada nova palavra, esse

resumo é atualizado. É, em essência, o modo como uma pessoa lê um texto em voz alta, guardando na memória o que veio antes. Dessa mecânica decorrem os dois problemas que motivaram o *Transformer*.

O primeiro é o esquecimento. Considere o trecho "A Inglaterra é minha terra natal. Passei toda a minha vida lá. Mudei-me para a Espanha há dois dias. Eu falo apenas uma língua, que é o ___". Preencher a lacuna com "inglês" exige recuperar uma palavra que apareceu muito antes. Como o resumo é reescrito a cada passo, a informação antiga vai se diluindo, e a influência de "Inglaterra" chega enfraquecida ao final. Tecnicamente, o fenômeno se manifesta durante o treinamento como gradiente desaparecente (*vanishing gradient*): os gradientes são multiplicados repetidamente pelas matrizes de peso e, quando esses valores são pequenos, encolhem até se aproximarem de zero (BENGIO; SIMARD; FRASCONI, 1994). É a limitação conhecida como incapacidade de lidar com dependências de longo alcance. Arquiteturas posteriores, como as redes LSTM (*Long Short-Term Memory*), atenuaram o esquecimento por meio de mecanismos que regulam o que deve ser mantido ou descartado (HOCHREITER; SCHMIDHUBER, 1997), mas não resolveram o segundo problema.

O segundo é a lentidão, como cada palavra só pode ser processada depois da anterior, a leitura é obrigatoriamente sequencial: não há como dividir o trabalho entre vários processadores. Isso desperdiça justamente a principal característica das unidades de processamento gráfico (GPUs) modernas, que é executar milhares de operações ao mesmo tempo.

Foi para atacar essas duas limitações simultaneamente que se desenvolveu o mecanismo de atenção.

2.3.2 Mecanismo de Atenção

O passo seguinte dessa evolução foi o mecanismo de atenção (*attention mechanism*), introduzido por Bahdanau, Cho e Bengio (2015) no contexto da tradução automática. Em vez de comprimir toda a frase em um único estado oculto, a atenção permite que o modelo "olhe" de volta e foque seletivamente em partes específicas da entrada. No exemplo anterior, ela pode consultar diretamente o termo "Inglaterra", independentemente de quão distante ele esteja da lacuna, eliminando o risco de o gradiente desaparecer pelo caminho.

Para operacionalizar essa ideia, o mecanismo utiliza três conceitos: a consulta (*query*), a chave (*key*) e o valor (*value*). Uma analogia útil é a de uma biblioteca: cada livro tem uma etiqueta na lombada (a *chave*) que descreve seu conteúdo; quando um leitor chega com uma pergunta (a *consulta*), ela é comparada com cada etiqueta para decidir quais livros são mais relevantes; o conteúdo desses livros (os *valores*) é então com-

binado, com os mais relevantes contribuindo mais para a resposta. No *Transformer*, esse processo ocorre com números: a relevância entre uma consulta e cada chave é calculada matematicamente, e os valores são somados com pesos proporcionais a essa relevância.

A inovação central de Vaswani et al. (2017), em 2017, foi construir uma arquitetura inteira baseada exclusivamente nesse mecanismo, dispensando por completo a recorrência. Com isso, todas as posições de uma sequência se comunicam diretamente em uma única operação, sem precisar "passar" por palavras intermediárias. Isso resolve as duas limitações das RNNs de uma só vez: as dependências de longo alcance tornam-se tão fáceis de capturar quanto as de curto alcance, e o cálculo passa a ser paralelizável, tornando as arquiteturas baseadas em RNN e LSTM obsoletas para muitas tarefas modernas.

2.3.3 Autoatenção e Atenção Multi-Cabeça

A variante do mecanismo de atenção utilizada no *Transformer* é chamada de autoatenção (*self-attention*). O prefixo "auto" indica que a consulta, a chave e o valor são todos derivados da mesma sequência de entrada. Ou seja, ao processar uma frase, cada palavra pergunta a si mesma: "quais outras palavras desta frase são relevantes para minha interpretação?". Exemplo: "O gato que estava no telhado dormiu". Ao processar a palavra "dormiu", o mecanismo de autoatenção calcula um peso para cada palavra da frase. A palavra "gato" recebe peso alto, pois é o sujeito do verbo; palavras como "telhado" ou "no" recebem pesos baixos, pois são menos relevantes para determinar quem dormiu.

Na prática, esses pesos são calculados da seguinte forma, a partir de cada representação de entrada, três matrizes de pesos aprendíveis durante o treinamento transformam cada representação em três vetores: o vetor de consulta Q (*query*), o vetor de chave K (*key*) e o vetor de valor V (*value*). O grau de compatibilidade entre uma consulta e cada chave é calculado pelo produto escalar entre eles. Esses valores brutos de compatibilidade passam pela função *softmax*, que os converte em uma distribuição de probabilidades, isto é, um conjunto de números entre 0 e 1 que somam exatamente 1. Esses pesos normalizados determinam com que intensidade cada valor V contribui para o resultado final, que é obtido pela soma ponderada dos valores.

Essa operação completa é denominada *Scaled Dot-Product Attention* (atenção por produto escalar escalado) e é definida formalmente por (VASWANI et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Em termos intuitivos, o produto $QK^\top$ mede a compatibilidade entre todas as consultas e todas as chaves de uma só vez; a divisão por $\sqrt{d_k}$ (a dimensão das chaves) é um fator de escala que estabiliza o treinamento; a *softmax* converte essas pontuações em pesos

que somam 1; e a multiplicação por V produz, para cada posição, uma média ponderada dos valores, isto é, sua representação já contextualizada pelo restante da sequência.

Uma única operação de atenção, porém, captura apenas um tipo de relação entre as palavras de cada vez. Para superar essa limitação, Vaswani et al. (2017) propõem a atenção multi-cabeça (*multi-head attention*). A analogia é a seguinte: se a atenção simples é como um único leitor relendo a frase em busca de relações gramaticais, a atenção multi-cabeça é como h leitores independentes, cada um procurando um tipo diferente de relação. Um leitor pode focar em concordância verbal ("gato"/"dormiu"), outro em relações de posse ("telhado do vizinho"), outro em referências pronominais, e assim por diante. O GPT-2 *small*, por exemplo, utiliza 12 cabeças de atenção por bloco.

Na prática, cada uma das h cabeças executa a operação *Scaled Dot-Product Attention* de forma independente, com suas próprias matrizes de projeção aprendíveis para Q, K e V. As saídas das h cabeças são depois concatenadas e projetadas por uma matriz linear aprendível, produzindo a representação final de cada posição. Desse modo, o modelo pode simultaneamente capturar relações em diferentes subespaços semânticos e sintáticos, sem custo adicional expressivo em relação à atenção simples.

Sob a perspectiva da compressão de modelos, as cabeças de atenção constituem unidades podáveis naturais para a poda estruturada: estudos da literatura demonstram que nem todas as cabeças são igualmente úteis, e que algumas podem ser removidas inteiramente sem degradação significativa da qualidade das predições (MICHEL; LEVY; NEUBIG, 2019).

2.3.4 Estrutura do Bloco Transformer

Para entender como o *Transformer* processa informação em profundidade, é útil pensar em cada bloco como uma "estação de trabalho" em uma linha de montagem. Na primeira bancada, as palavras trocam informações entre si por meio da atenção multi-cabeça: cada palavra atualiza sua representação consultando todas as demais. Na segunda bancada, cada palavra refina individualmente sua própria representação, de forma independente das outras, por meio de uma pequena rede totalmente conectada. Esse par de operações se repete em cada bloco, e os blocos são empilhados, fazendo com que as representações das palavras se tornem progressivamente mais ricas e abstratas a cada andar.

O primeiro bloco é a camada de atenção multi-cabeça, já apresentada na seção anterior, responsável por integrar informação contextual de toda a sequência. O segundo é uma rede *feedforward* totalmente conectada — um MLP (*multilayer perceptron*) aplicado de forma idêntica e independente a cada posição. Esse MLP é composto por duas transformações lineares com uma função de ativação não linear entre elas: a função GELU

(*Gaussian Error Linear Unit*) (HENDRYCKS; GIMPEL, 2016) nos modelos GPT, ou ReLU em outras variantes. Diferentemente da ReLU (NAIR; HINTON, 2010), que simplesmente zera valores negativos, a GELU faz uma transição suave em torno da origem. Na prática, essa suavidade evita o problema dos "neurônios mortos" (*dying ReLU*), em que unidades deixam de aprender por receberem gradiente nulo, e por isso Radford et al. (2019) a adotaram como ativação padrão na família GPT. A dimensão intermediária desse MLP é tipicamente quatro vezes maior do que a dimensão do modelo — no GPT-2 *small*, a dimensão do modelo é 768 e a dimensão intermediária é 3.072 — o que confere ao bloco grande capacidade de representação antes de comprimir novamente a informação.

2.3.5 Modelos de Linguagem de Grande Escala e o GPT-2

Modelos de linguagem de grande escala (*Large Language Models*, LLMs) são redes neurais profundas, tipicamente baseadas na arquitetura *Transformer*, treinadas em grandes volumes de texto com o objetivo de aprender a prever palavras. A tarefa de pré-treinamento é simples de enunciar: dado um trecho de texto, o modelo deve prever qual token vem a seguir. Um *token* é a unidade mínima de texto que o modelo processa — pode ser uma palavra, parte de uma palavra, um sinal de pontuação ou um espaço. Dada a sequência "O aluno abriu o", um LLM bem treinado atribuiria alta probabilidade a tokens como "livro", "caderno" ou "computador", e baixa probabilidade a tokens como "foguete" ou "plutônio". Repetida bilhões de vezes sobre textos variados, essa tarefa aparentemente simples força o modelo a aprender gramática, fatos sobre o mundo, raciocínio básico e muito mais (RADFORD et al., 2019).

Para que o texto possa ser processado numericamente, ele precisa primeiro ser convertido em tokens e depois em vetores numéricos. Essa conversão ocorre em duas etapas. A primeira é a tokenização, realizada por algoritmos como o BPE (*Byte Pair Encoding*), que divide o texto em unidades de comprimento variável de forma a equilibrar cobertura e eficiência. Por exemplo, a palavra "inconstitucional" pode ser dividida em tokens como "in", "constitu" e "cional". O GPT-2 possui um vocabulário de aproximadamente 50.257 tokens. A segunda etapa converte cada token em um vetor numérico denso por meio de uma camada de *embedding*: o modelo aprende, durante o treinamento, uma representação vetorial para cada token do vocabulário, de modo que tokens semanticamente próximos tendam a ter representações próximas no espaço vetorial.

O GPT-2 (*Generative Pre-trained Transformer 2*), proposto por Radford et al. (2019) pela OpenAI em 2019, é um modelo autorregressivo baseado exclusivamente no componente decodificador do *Transformer* original. Ele foi pré-treinado em uma base de dados de aproximadamente 40 GB de texto coletado da internet, abrangendo uma grande diversidade de domínios e estilos. A variante GPT-2 *small*, utilizada neste trabalho, possui 12 blocos *Transformer* empilhados, 12 cabeças de atenção por bloco, dimensão de

modelo igual a 768 e aproximadamente 124 milhões de parâmetros treináveis. Apesar de ser a menor variante da família GPT-2, esse número de parâmetros é cerca de cinco vezes maior do que o de uma ResNet-50 (25 milhões de parâmetros), arquitetura de referência nos estudos clássicos de poda — e as variantes maiores da própria família GPT-2 chegam a 1,5 bilhão de parâmetros, o que ilustra a escala característica dos modelos baseados em *Transformer*.

O crescimento em escala dos LLMs ao longo dos anos seguintes foi vertiginoso: o GPT-3, publicado em 2020, possui 175 bilhões de parâmetros; o PaLM, do Google, chega a 540 bilhões. Mesmo modelos de médio porte como o GPT-2 impõem custos energéticos e computacionais consideráveis em ambientes de produção, onde a inferência é executada milhares ou milhões de vezes por dia. A poda estruturada surge, nesse contexto, como uma estratégia concreta para reduzir esses custos: a remoção de cabeças de atenção redundantes e de neurônios MLP de baixa importância pode diminuir substancialmente a contagem de operações por inferência, com impacto controlado sobre a qualidade das predições (PATTERSON et al., 2021).

2.4 Consumo Energético em Redes Neurais

O avanço de técnicas e *hardware* voltados ao treinamento de redes neurais, motivado pela busca por maior precisão, trouxe consigo um consumo de energia considerável e, consequentemente, uma significativa emissão de dióxido de carbono. Strubell, Ganesh e McCallum (2019) relatam, por exemplo, que o treinamento de um modelo *transformer* com busca de arquitetura neural pode emitir cerca de 626.155 libras de CO_2, enquanto a emissão média de um automóvel ao longo de toda a sua vida útil é de aproximadamente 126.000 libras — cerca de cinco vezes menos.

Esse custo energético concentra-se em duas etapas do ciclo de vida de um modelo: o treinamento e a inferência.

No treinamento, o modelo é alimentado com grandes volumes de dados e ajustado iterativamente pelo *backpropagation*. Esse processo é computacionalmente intenso e geralmente requer *hardware* especializado (GPUs ou TPUs), muitas vezes organizado em *clusters* com centenas ou milhares de aceleradores (PATTERSON et al., 2021; SCHWARTZ et al., 2020).

Na inferência, o modelo já treinado é utilizado para gerar predições. O custo de uma única predição é relativamente baixo; contudo, em contextos de larga escala — como os modelos de IA generativa servindo milhões de requisições diárias — o custo energético agregado é expressivo (LUCCIONI; VIGUIER; LIGOZAT, 2022).

A diferença entre as duas etapas é de natureza, não apenas de grandeza. O trei-

namento é um gasto concentrado e único, comparável ao custo de construir uma ponte; a inferência é um gasto pequeno, porém perpétuo, comparável ao custo de mantê-la aberta ao tráfego. Uma ponte muito usada acaba custando, ao longo de décadas, mais em manutenção do que custou para ser erguida — e o mesmo ocorre com um modelo consultado milhões de vezes por dia. É por essa razão que este trabalho concentra-se na eficiência da inferência.

2.5 Compressão de Modelos

Com o crescimento do *Deep Learning*, os modelos de IA vêm dobrando em tamanho a cada poucos anos (SEVILLA et al., 2022). Modelos de alta performance costumam ser otimizados exclusivamente para acurácia, resultando em redes com grande quantidade de parâmetros, que ocupam muito espaço de armazenamento e demandam um elevado número de operações em ponto flutuante (FLOPs) para serem executadas. Isso é particularmente problemático em dispositivos com capacidade limitada, como *smartphones* e sistemas embarcados, e também em contextos de uso em larga escala, nos quais o custo energético se torna significativo.

A compressão de modelos é um campo que reúne técnicas voltadas a atenuar esses problemas, reduzindo o tamanho dos modelos e/ou a quantidade de operações necessárias para sua execução, preservando, tanto quanto possível, a acurácia. As redes neurais modernas possuem alta redundância de parâmetros (DENIL et al., 2013), o que torna viável remover ou simplificar aqueles com baixa contribuição ao resultado final.

Essa redundância não é um defeito, mas uma consequência do modo como esses modelos são construídos. Como não se sabe de antemão quantos parâmetros a tarefa exige, projeta-se a rede com folga generosa, do mesmo modo que se dimensiona uma instalação elétrica para uma carga muito acima da que será efetivamente usada. A folga facilita o treinamento, mas permanece no modelo depois que ele já aprendeu — e é justamente essa sobra que as técnicas de compressão procuram eliminar.

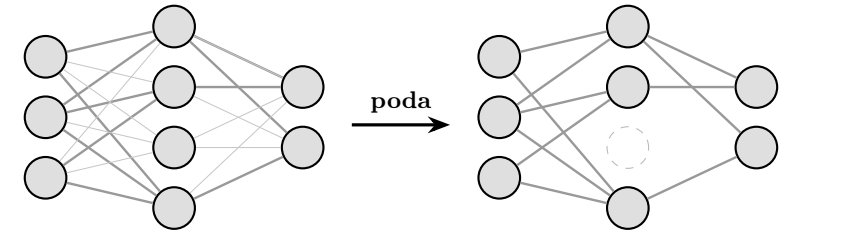
As quatro principais famílias de técnicas de compressão são:

- **Quantização (Quantization):** reduz o número de *bits* utilizados para representar cada peso do modelo. Em vez de pontos flutuantes de 32 bits, os parâmetros passam a ser representados com precisão reduzida (16 ou 8 bits, por exemplo), o que diminui o tamanho do modelo e pode acelerar operações em *hardware* que suporta aritmética de menor precisão (JACOB et al., 2018). O equivalente cotidiano é anotar preços arredondados: registrar R$ 20 em vez de R$ 19,99 ocupa menos espaço e agiliza as contas, e na maioria das situações a diferença é irrelevante.
- **Destilação de Conhecimento (Knowledge Distillation):** proposta por Hinton,

Vinyals e Dean (2015), é uma metodologia de treinamento em que um modelo menor (*aluno*) aprende a partir de um modelo maior e mais preciso (*professor*). O aluno é treinado não apenas para acertar os rótulos verdadeiros, mas também para imitar a distribuição de saída do professor, resultando em um modelo compacto que, frequentemente, alcança acurácia superior à que teria se fosse treinado do zero.

- **Fatoração de Matrizes (Matrix Factorization):** matrizes de pesos de grandes dimensões são decompostas em produto de matrizes menores cuja multiplicação aproxima a matriz original, reduzindo o custo computacional (DENTON et al., 2014).
- **Poda Neural (Neural Pruning):** processo de remoção de pesos — ou de estruturas maiores, como filtros — que contribuem pouco para a saída do modelo. É a técnica central deste trabalho e será detalhada na Seção a seguir.

Figura 3 – Ilustração do processo de poda: a rede original (densa) tem conexões e/ou neurônios removidos, tornando-se esparsa.



Fonte: Adaptado de Han et al. (2015).

2.6 Poda Neural (Pruning)

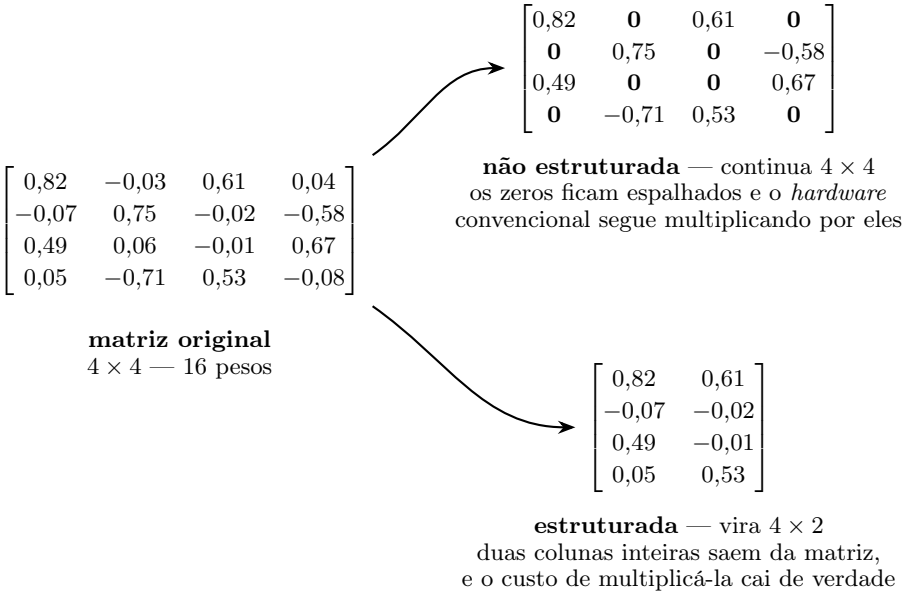
A poda neural é uma técnica de compressão que visa reduzir o tamanho do modelo e o custo computacional da inferência por meio da remoção de parâmetros que pouco contribuem para o resultado final. O processo mais comum parte de um modelo previamente treinado, cujo foco inicial foi exclusivamente a acurácia, e aplica a poda como uma etapa posterior (HAN et al., 2015; BLALOCK et al., 2020).

2.6.1 Tipos de Poda

Na literatura, duas categorias principais de poda são consideradas (BLALOCK et al., 2020):

- **Poda não estruturada:** remoção de pesos individuais, independentemente da sua posição na matriz de pesos. Esta abordagem oferece maior flexibilidade e, geralmente, taxas de compressão elevadas com perda reduzida de acurácia. Entretanto, do ponto de vista do custo de inferência, a poda não estruturada apresenta uma limitação importante: processadores e GPUs convencionais não são eficientes em pular multiplicações por zero espalhadas de forma aleatória na matriz de pesos. Assim, embora o modelo tenha muitos zeros, as operações ainda são executadas, e o ganho prático de velocidade e energia tende a ser marginal, a menos que se utilize *hardware* especializado ou formatos de armazenamento esparso.
- **Poda estruturada:** remoção de estruturas inteiras, como neurônios, filtros ou canais convolucionais, o que implica a remoção completa de detectores de características. Como o resultado é uma matriz menor e ainda densa, há um ganho direto de velocidade e eficiência em *hardware* convencional. O custo dessa abordagem é uma potencial perda maior de acurácia para uma mesma taxa de compressão.

Figura 4 – As duas categorias de poda aplicadas à mesma matriz de pesos. Na poda não estruturada os pesos removidos são apenas zerados e a matriz conserva suas dimensões; na poda estruturada, colunas inteiras são eliminadas e a matriz encolhe de fato.



A distinção entre as duas categorias é o que separa um ganho real de um ganho apenas contábil, e uma analogia com um livro a torna evidente. A poda não estruturada equivale a apagar palavras espalhadas por todas as páginas: o texto fica mais enxuto, mas o livro continua com o mesmo número de páginas, e quem for lê-lo ainda terá de folhear todas elas — inclusive as partes apagadas. A poda estruturada equivale a arrancar capítulos inteiros: o livro encolhe de fato, pesa menos e é percorrido em menos tempo. Como o *hardware* convencional não sabe pular um espaço em branco no meio de uma

multiplicação, apenas a segunda forma se converte em redução efetiva de tempo e de energia. Essa diferença é retomada no Capítulo 6, onde é verificada experimentalmente.

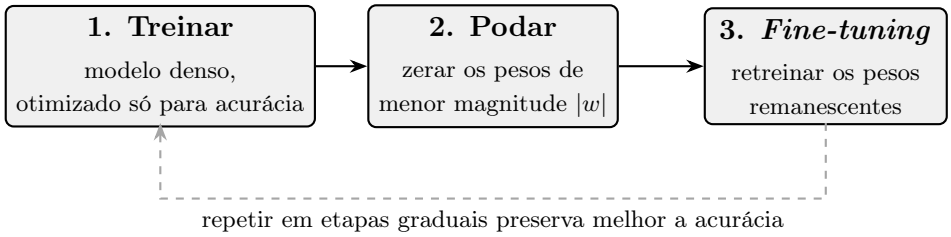
Cabe mencionar também a chamada esparsidade estruturada (por exemplo, padrões 2:4), em que o algoritmo é forçado a zerar, a cada bloco de quatro pesos, exatamente dois. Como o *hardware* sabe previamente quantos e em que formato os zeros aparecem, a matriz pode ser comprimida de maneira determinística, permitindo acelerações reais em GPUs modernas que suportam esse padrão (NVIDIA Corporation, 2020).

Assim, a poda estruturada é a que apresenta benefícios mais imediatos em termos de FLOPs e consumo de energia em *hardware* convencional (LI et al., 2017), razão pela qual ela recebe destaque neste trabalho.

2.6.2 Poda por Magnitude

A técnica mais simples e amplamente estudada é a poda por magnitude. Nela, considera-se que pesos cujos valores absolutos são próximos de zero contribuem pouco para a saída do modelo e, portanto, podem ser removidos. Define-se um limiar (ou uma taxa de esparsidade desejada) e todos os pesos abaixo desse limiar são zerados, criando redes esparsas (HAN et al., 2015).

Figura 5 – Pipeline clássico de poda por magnitude: treinamento do modelo denso, remoção dos pesos de menor magnitude e retreinamento (*fine-tuning*) dos pesos remanescentes.



Fonte: Adaptado de Han et al. (2015).

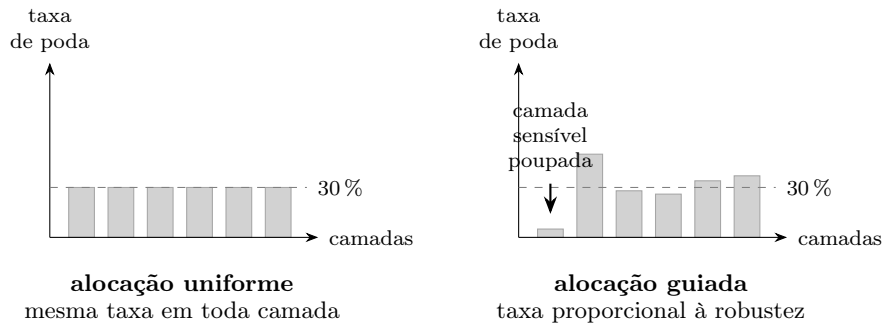
2.6.3 Sensibilidade e a Alocação do Orçamento de Poda

A poda por magnitude responde a uma pergunta: *quais* pesos remover. Ela deixa intocada uma segunda pergunta, logicamente anterior e menos discutida: *quanto* remover de cada parte da rede. Um modelo com doze camadas e um orçamento de poda de 30 % admite infinitas distribuições desse orçamento — 30 % em cada camada, 60 % na metade delas e nada na outra metade, ou qualquer combinação intermediária. Todas removem a mesma quantidade de parâmetros e produzem modelos de custo praticamente idêntico, mas não produzem modelos de qualidade idêntica.

A analogia é a de uma empresa que precisa cortar 30 % das despesas. O corte linear — 30 % em todos os setores — é o mais simples de justificar e o que exige menos informação, mas trata a contabilidade e a produção como igualmente dispensáveis. Um corte melhor exige saber de antemão qual setor a empresa suporta perder e qual a paralisa. Essa informação não está no orçamento de cada setor: um setor caro pode ser supérfluo, e um setor barato pode ser crítico. Ela só aparece quando se pergunta o que acontece com a empresa na ausência de cada setor.

É exatamente essa a definição de **sensibilidade** adotada neste trabalho: a sensibilidade de uma camada é o quanto o desempenho do modelo se degrada quando aquela camada, e somente ela, é podada. Trata-se de uma medida empírica e direta de importância, obtida por ablação, e não de um *proxy*. A distinção em relação à magnitude é central: a magnitude é uma propriedade dos pesos, disponível sem custo algum, mas apenas correlacionada com a importância; a sensibilidade é uma propriedade medida do comportamento do modelo, cara de obter, porém direta.

Figura 6 – Duas distribuições do mesmo orçamento de poda entre as camadas de uma rede. À esquerda, a alocação uniforme, que ignora qualquer medida de importância; à direita, a alocação guiada por sensibilidade, que poupa as camadas mais sensíveis e compensa nas mais robustas. Ambas removem a mesma fração total de parâmetros. Figura esquemática.



A ideia de medir importância por degradação não é nova. Ela remonta ao *Optimal Brain Damage* de LeCun, Denker e Solla (1990), que estima, por aproximação de segunda ordem, o aumento de erro provocado pela remoção de cada peso, e ordena os parâmetros por essa estimativa em vez de por magnitude. A limitação do método é o custo: a aproximação exige informação de curvatura, cujo cálculo é proibitivo em redes com centenas de milhões de parâmetros. A alternativa adotada aqui troca a estimativa analítica por medição direta e reduz drasticamente a granularidade — em vez de estimar a importância de cada peso individualmente, mede-se a de cada camada. Com doze camadas, o perfil completo custa doze avaliações do modelo, sem nenhum treinamento envolvido.

Há evidência consolidada de que essa desigualdade entre partes da rede existe e é substancial. Denil et al. (2013) mostram que uma fração pequena dos parâmetros de uma rede é suficiente para predizer todos os demais, o que indica redundância distribuída

de forma desigual. Em *Transformers* especificamente, Michel, Levy e Neubig (2019) verificam que a maioria das cabeças de atenção pode ser removida após o treinamento com perda pequena de desempenho, enquanto algumas poucas são indispensáveis — e que quais delas são indispensáveis não é previsível a partir de propriedades dos pesos. Se a importância fosse aproximadamente uniforme, a alocação do orçamento seria irrelevante e a poda uniforme seria ótima por construção; é precisamente porque não é uniforme que existe espaço para alocar melhor.

Duas ressalvas conceituais acompanham o método e delimitam o que ele pode oferecer.

A primeira é que a sensibilidade medida por ablação de uma camada por vez não captura interações entre camadas. A degradação causada por podar duas camadas simultaneamente não é, em geral, a soma das degradações individuais, de modo que o perfil deve ser tratado como uma **ordenação relativa de robustez**, e não como um modelo capaz de prever a qualidade de uma configuração completa. A segunda é que a sensibilidade não é uma propriedade absoluta da camada: ela é medida em relação a uma estratégia de poda específica, porque remover pesos de menor magnitude e remover cabeças de atenção inteiras são operações diferentes, que danificam a camada de maneiras diferentes. O perfil precisa, portanto, ser levantado uma vez para cada estratégia.

O procedimento que converte esse perfil em taxas de poda por camada é detalhado na Seção 5.4, e ambas as ressalvas acima são verificadas experimentalmente no Capítulo 6.

2.6.4 Pipeline Geral da Poda

De forma geral, a poda pode ser dividida nos seguintes passos (BLALOCK et al., 2020):

- **Passo 1 – Definição de critério e taxa de poda:** partindo de um modelo já treinado, define-se o critério de importância dos parâmetros (magnitude absoluta, norma L_1 de filtros, sensibilidade, entre outros) e a taxa de esparsidade desejada.
- **Passo 2 – Identificação dos parâmetros a podar:** com base no critério escolhido, o algoritmo classifica os parâmetros por importância e seleciona os menos relevantes.
- **Passo 3 – Execução da poda:** os parâmetros selecionados são zerados (no caso da poda não estruturada) ou fisicamente removidos (no caso da poda estruturada), gerando um modelo esparso ou uma nova arquitetura reduzida.
- **Passo 4 – Ajuste fino (*fine-tuning*):** como a remoção tende a deteriorar a acurácia, o modelo podado é retreinado com uma taxa de aprendizado reduzida, para que os parâmetros remanescentes compensem a ausência dos que foram removidos.

- **Passo 5 – Iteração:** a poda pode ser aplicada de forma iterativa, em pequenas etapas intercaladas por *fine-tuning*, o que costuma preservar melhor a acurácia do que uma poda agressiva em um único passo.

2.6.5 Quando a Poda é Aplicada

Tradicionalmente, a poda é aplicada após o treinamento inicial do modelo, seguida de um ajuste fino. Métodos mais recentes aplicam a poda de forma gradual durante o próprio treinamento, intercalando passos de treino e poda, o que permite à rede adaptar-se progressivamente à nova estrutura (ZHU; GUPTA, 2017).

Uma linha de pesquisa mais recente investiga a possibilidade de podar a rede antes do treinamento, identificando sub-redes inerentemente promissoras dentro de uma rede densa inicializada aleatoriamente. Essa abordagem é impulsionada pela chamada *Lottery Ticket Hypothesis* (FRANKLE; CARBIN, 2019), segundo a qual, dentro de uma rede grande inicializada aleatoriamente, existe uma sub-rede pequena (o "bilhete premiado") que, quando treinada isoladamente com a mesma inicialização, atinge acurácia comparável à da rede completa.

3 Trabalhos Relacionados

Este capítulo apresenta trabalhos da literatura diretamente relacionados ao tema desta pesquisa, com foco em estratégias de poda neural e, em especial, naquelas que visam a redução do consumo energético de modelos de *Deep Learning*. Para cada trabalho, são descritos os objetivos principais, a metodologia adotada e os principais resultados obtidos.

3.1 Designing Energy-Efficient Convolutional Neural Networks using Energy-Aware Pruning

Yang, Chen e Sze (2017) propõem um algoritmo de poda para redes neurais convolucionais (CNNs) cujo foco é minimizar o consumo real de energia, e não apenas o número de parâmetros ou de operações, preservando a acurácia. Os autores consideram especialmente o custo computacional associado a operações de multiplicação e acumulação (MAC) e o custo de acesso à memória em dispositivos periféricos, como *smartphones*.

O método proposto aplica poda camada por camada, priorizando aquelas cujo consumo energético estimado é maior. Para isso, os autores desenvolvem um *framework* de estimativa que ordena as camadas pela combinação de custo computacional (número de MACs) e custo de acesso à memória. Em cada camada podada, parte dos pesos é removida; em seguida, alguns pesos são restaurados para minimizar o erro no mapa de características de saída; por fim, os pesos remanescentes são ajustados localmente por meio de uma solução de mínimos quadrados. Após o processamento de todas as camadas, a rede passa por um *fine-tuning* global utilizando *backpropagation*.

Os autores reportam redução de 3,7× no consumo de energia da AlexNet e 1,6× no da GoogLeNet, com perda de acurácia inferior a 1%. O trabalho também demonstra empiricamente que as camadas convolucionais são as que mais consomem energia, principalmente devido ao volume de acessos à memória, e que a simples redução do número de classes tem impacto limitado na redução do consumo energético, uma vez que as camadas convolucionais não são significativamente afetadas.

A fazer: adicionar resenhas de outros trabalhos fundamentais para esta pesquisa. Sugestões: Han et al. (2015) (poda por magnitude não estruturada – seminal); Li et al. (2017) (poda estruturada L1 em filtros – seminal); Frankle e Carbin (2019) (Lottery Ticket Hypothesis); Blalock et al. (2020) (revisão crítica de métodos de poda); Liu et al. (2019) (*Rethinking the Value of Network Pruning*). Para cada um, seguir o mesmo padrão de resenha: objetivo, método, resultados principais.

4 Metodologia

Este capítulo apresenta a metodologia adotada neste trabalho, organizada em quatro etapas sequenciais:

- **Etapa 1 — Modelo de Referência** (Seção 4.1): seleção do modelo de referência e da base de dados sobre a qual ele é avaliado, e definição das métricas registradas para esse modelo e para todos os modelos podados;
- **Etapa 2 — Poda sem Otimização** (Seção 4.2): aplicação de estratégias clássicas de poda ao modelo de referência;
- **Etapa 3 — Poda com Otimização** (Seção 4.3): aplicação das mesmas estratégias combinadas a uma técnica de otimização, que redistribui a poda entre as camadas do modelo;
- **Etapa 4 — *Fine-tuning* de Recuperação** (Seção 4.4): retreinamento dos modelos podados, para medir quanto da degradação causada pela poda pode ser recuperada.

Cada etapa é descrita aqui de forma genérica: o que nela se faz e os critérios que orientam suas decisões. As decisões concretas, o modelo e a base de dados selecionados, as configurações de poda e os parâmetros do retreinamento, são apresentadas no Capítulo 5, organizado nas mesmas quatro etapas, na mesma ordem.

4.1 Etapa 1 — Modelo de Referência

Na primeira etapa, seleciona-se o modelo de referência (*baseline*), a versão original de um modelo de linguagem de grande escala (*Large Language Model* — LLM), antes de qualquer processo de poda. O modelo é executado sem nenhuma modificação e avaliado sobre uma base de dados, registrando-se métricas de desempenho preditivo, de custo computacional e de consumo de energia. Os valores obtidos constituem o referencial de comparação de todo o trabalho, as métricas de cada modelo podado nas etapas seguintes são comparadas às desse modelo original, o que permite mensurar a perda de desempenho e o ganho de eficiência introduzidos pela compressão.

A etapa depende, portanto, de três definições, apresentadas a seguir: o modelo, a base de dados sobre a qual ele é avaliado e as métricas registradas.

4.1.1 Seleção do Modelo

Para a seleção do modelo de referência, foram adotados os seguintes critérios:

- **C1 — Arquitetura:** ser um *Transformer* composto apenas pelo componente decodificador, treinado de forma autorregressiva para a predição do próximo *token* (Seção 2.3.5). A representatividade é definida pela classe arquitetural: a mesma de modelos generativos de pesos abertos como OPT e Llama 2 (ZHANG et al., 2022; TOUVRON et al., 2023), e a classe para a qual a *perplexity* é diretamente definida como métrica de desempenho (Hugging Face, 2026). Como indicador complementar de adoção, registra-se para cada candidato o número de *downloads* na plataforma Hugging Face (Tabela 3).

- **C2 — Pesos abertos:** dispor de pesos pré-treinados com *download* direto na plataforma que os hospeda, sem exigência de aprovação prévia de acesso, sob licença que permita uso acadêmico. O pré-treinamento a partir do zero está fora do alcance deste trabalho: exige bases de dados da ordem de dezenas de gigabytes de texto — o WebText, empregado no pré-treinamento da família GPT-2, reúne pouco mais de 8 milhões de documentos e 40 GB de texto (RADFORD et al., 2019) — e tem custo computacional e energético elevado (STRUBELL; GANESH; MCCALLUM, 2019).

- **C3 — Viabilidade do *fine-tuning*:** a memória exigida pelo *fine-tuning* completo do modelo deve caber na GPU disponível. Como estimativa, o treinamento com o otimizador Adam em precisão mista exige, no mínimo, 16 bytes por parâmetro apenas para pesos, gradientes e estados do otimizador (RAJBHANDARI et al., 2020); as ativações somam-se a esse valor, de modo que a estimativa constitui um limite inferior de memória necessária. No ambiente adotado — GPU NVIDIA Tesla T4, com 16 GB de memória (NVIDIA Corporation, 2019), disponibilizada pela plataforma Kaggle —, o critério restringe os candidatos a modelos de até cerca de 1 bilhão de parâmetros (16 GB $\div$ 16 bytes por parâmetro); como a estimativa não inclui as ativações, o limite prático é inferior.

- **C4 — Modelo original:** não ser derivado de um processo prévio de compressão (destilação, poda ou quantização), de modo que a degradação medida ao longo deste trabalho seja atribuível exclusivamente à poda aqui aplicada.

- **C5 — Resultado de referência:** ter *perplexity* na base de dados de avaliação adotada publicada em artigo científico — pelos autores do modelo ou em trabalho posterior, o que permite situar o modelo de referência reproduzido em relação à literatura.

- **C6 — Implementação disponível:** possuir arquitetura implementada nativamente na biblioteca `transformers` (WOLF et al., 2020), que opera sobre a biblioteca PyTorch, dispensando a implementação própria da arquitetura.

Os critérios C1 a C4 e C6 dizem respeito apenas ao modelo. O critério C5, ao contrário, depende também da base de dados de avaliação, cuja seleção segue critérios próprios (Seção 4.1.2), dois dos quais, D1 e D4, dependem, por sua vez, do modelo. Por essa razão, modelo e base de dados são selecionados em conjunto: um par formado por modelo e base de dados é admissível quando o modelo atende a C1–C6 e a base de dados atende a D1–D4 em relação a esse modelo.

Havendo mais de um candidato que atenda simultaneamente a todos os critérios, adota-se o de menor número de parâmetros: as varreduras experimentais deste trabalho exigem um grande número de avaliações completas do modelo, e o custo computacional de cada avaliação cresce com o tamanho do modelo. A aplicação desses critérios a um conjunto de modelos candidatos é apresentada no Capítulo 5 (Tabela 3).

4.1.2 Seleção da Base de Dados

O modelo de referência e todos os modelos podados nas etapas seguintes são avaliados sobre uma mesma base de dados — a coleção de textos sobre a qual se mede o desempenho dos modelos —, selecionada pelos seguintes critérios:

- **D1 — Resultado de referência:** *perplexity* do modelo de referência na base de dados publicada em artigo científico; é a exigência do critério C5 vista do lado da base de dados.
- **D2 — Texto sem pré-processamento destrutivo:** a base de dados preserva caixa, pontuação, números e palavras raras, e mantém a continuidade dos documentos; modelos com tokenização por subpalavras são pré-treinados sobre texto bruto e seriam avaliados fora de sua distribuição em texto normalizado.
- **D3 — Divisão oficial:** partições oficiais de treino, validação e teste, de modo que o *fine-tuning* use o treino e a avaliação use o teste, sem particionamento arbitrário.
- **D4 — Ausência de sobreposição com o pré-treinamento:** a base de dados não integra nem reproduz os dados de pré-treinamento do modelo; do contrário, a *perplexity* mediria em parte memorização.

Os critérios D2 e D3 dizem respeito apenas à base de dados; D1 e D4 dependem do modelo, e por isso modelo e base de dados são selecionados em conjunto (Seção 4.1.1). Havendo mais de uma base de dados que atenda a todos os critérios, adota-se a de

menor conjunto de treino, pois é ele que determina o custo do *fine-tuning*. A aplicação desses critérios a um conjunto de bases de dados candidatas é apresentada no Capítulo 5 (Tabela 4).

4.1.3 Métricas de Avaliação

As métricas de avaliação são organizadas em três dimensões: desempenho preditivo, custo computacional e impacto ambiental. Elas são registradas pelo mesmo procedimento, e sobre a mesma base de dados, para o modelo de referência e para cada modelo podado nas Etapas 2 a 4. Para cada métrica, descreve-se o que ela representa e como é obtida operacionalmente.

Desempenho preditivo

Como o modelo de referência é um modelo de linguagem autorregressivo, a métrica de desempenho preditivo adotada é a *perplexity*, padrão para essa classe de modelos. Ela quantifica o grau de "surpresa" do modelo diante de cada *token* do conjunto de avaliação, constituindo uma medida da qualidade da modelagem probabilística da linguagem (GOODFELLOW; BENGIO; COURVILLE, 2016). Valores mais baixos de *perplexity* indicam melhor capacidade preditiva.

A adoção da *perplexity*, em lugar de métricas de tarefas descendentes — como a acurácia em uma tarefa de classificação —, decorre de ela ser uma métrica intrínseca: pode ser medida diretamente sobre a base de dados de avaliação, sem exigir o *fine-tuning* do modelo em uma tarefa específica, procedimento que tornaria a comparação entre configurações dependente de um segundo treinamento. O procedimento de cálculo adotado é descrito no Capítulo 5.

Custo computacional

O custo computacional é avaliado por meio de quatro indicadores complementares:

- **Número de parâmetros remanescentes:** contagem direta dos tensores de pesos não-zerados (no caso de poda não estruturada) ou existentes (no caso de poda estruturada) no modelo após a aplicação da técnica;
- **Operações de ponto flutuante (FLOPs):** contagem das operações matemáticas necessárias para uma inferência completa, obtida por análise estática do grafo computacional do modelo: ferramentas de contagem percorrem cada camada do modelo e acumulam as operações correspondentes. A automação, porém, é parcial: a contagem cobre apenas os tipos de camada previstos no catálogo da ferramenta, de modo

que camadas fora dele exigem o registro de regras de contagem próprias, sob pena de serem silenciosamente ignoradas. O tratamento efetivamente necessário no modelo adotado, bem como as omissões remanescentes, são detalhados no Capítulo 5;

- **Tempo de inferência:** medido empiricamente, registrando o tempo decorrido durante o processamento de uma entrada representativa em condições controladas de *hardware*. Múltiplas medições são agregadas por meio de média e desvio-padrão para mitigar a variância natural de execução;
- **Tamanho do modelo:** memória ocupada pelos parâmetros, calculada como o produto entre o número de parâmetros e o número de *bytes* usado para representar cada um.

Impacto ambiental

O impacto ambiental é mensurado por meio de dois indicadores:

- **Consumo energético:** estimado por meio de ferramentas de monitoramento do uso de *hardware* (CPU e GPU), que integram leituras periódicas do consumo elétrico dos componentes e registram a energia total consumida (em kWh) durante o intervalo de inferência;
- **Emissões equivalentes de CO_2:** calculadas a partir do consumo energético registrado, multiplicado por um fator de intensidade carbônica da matriz energética da região onde os experimentos são executados. As mesmas ferramentas de monitoramento mantêm bases desses fatores e realizam essa conversão automaticamente. Cabe uma ressalva sobre o alcance desse indicador: como o fator de intensidade carbônica depende da localidade em que o código executa, ele não é controlado pelo experimento e pode variar entre execuções. As emissões devem, por isso, ser lidas como aproximações de ordem de grandeza, e a comparação entre configurações apoia-se no consumo energético, não nas emissões dele derivadas.

4.2 Etapa 2 — Poda sem Otimização

Na segunda etapa, aplicam-se ao modelo de referência estratégias clássicas de poda, sem nenhuma técnica de otimização. Cada estratégia, em cada taxa de poda avaliada, é aplicada de forma independente e sempre a partir do modelo de referência original, nunca de um modelo já podado, de modo que o resultado de uma configuração não dependa das anteriores. Os modelos podados são avaliados pelas mesmas métricas e sobre a mesma base de dados da Etapa 1 (Seção 4.1).

As estratégias de poda sem otimização têm por objetivo reduzir o número de parâmetros de um modelo pré-treinado, eliminando pesos ou estruturas avaliadas como menos relevantes para o desempenho preditivo. A relevância é estimada por um critério de importância, que atribui uma pontuação a cada parâmetro ou a cada unidade arquitetural removível; os elementos de menor pontuação são removidos até que se atinja a taxa de poda pretendida.

As estratégias desta etapa não constituem a contribuição deste trabalho: desempenham o papel de linha de base. São aplicadas com os critérios de importância propostos por seus autores, e os modelos que produzem servem de referência para avaliar a estratégia com otimização (Seção 4.3), que constitui a contribuição central do trabalho.

A escolha das estratégias, entre as alternativas disponíveis na literatura, seguiu três critérios:

- **P1 — Importância calculada apenas a partir dos pesos:** o critério de importância não utiliza dados de calibração nem gradientes. Neste trabalho, a informação dependente de dados entra na decisão de poda exclusivamente pela análise de sensibilidade por camada (Seção 4.3), cuja contribuição se pretende medir; um critério-base que já utilizasse dados misturaria as duas fontes de informação e impediria atribuir à alocação por sensibilidade a diferença observada.
- **P2 — Aplicação *one-shot*:** a poda é aplicada ao modelo pré-treinado sem treinamento durante o processo, de modo que o efeito da remoção seja medido separadamente do efeito do *fine-tuning* de recuperação (Seção 4.4).
- **P3 — Cobertura das granularidades:** o conjunto de estratégias inclui uma de granularidade fina, que remove parâmetros individuais, e uma de granularidade grossa, que remove unidades arquiteturais inteiras.

O critério P3 decorre de as duas granularidades oferecerem vantagens opostas (Seção 2.6, Figura 4). A poda de granularidade fina alcança, em geral, taxas de compressão elevadas com perda reduzida de desempenho preditivo (BLALOCK et al., 2020); os pesos removidos, porém, são apenas anulados: as matrizes conservam suas dimensões, o *hardware* convencional continua a executar as multiplicações por zero, e a redução do número de parâmetros não se converte em redução de tempo nem de energia. A poda de granularidade grossa produz matrizes menores e densas, o que reduz de fato o custo de inferência em *hardware* convencional (LI et al., 2017), ao preço de uma perda potencialmente maior de desempenho preditivo para uma mesma taxa de compressão (BLALOCK et al., 2020). Uma estratégia preserva melhor o desempenho preditivo; a outra converte a compressão em economia efetiva de tempo e de energia. Avaliá-las sob as mesmas con-

dições permite medir esse compromisso, que nenhuma das duas revelaria isoladamente e que constitui uma das perguntas do trabalho (Seção 5.1).

A aplicação desses critérios a um conjunto de estratégias candidatas, as estratégias selecionadas e os detalhes de sua implementação são apresentados no Capítulo 5 (Tabela 6).

4.3 Etapa 3 — Poda com Otimização

Esta é a etapa central do trabalho. O objetivo é investigar se a combinação de uma estratégia clássica de poda com uma técnica de otimização resulta em modelos mais eficientes do que a poda aplicada de forma direta, sem qualquer refinamento adicional.

A hipótese central do trabalho é a de que as camadas do modelo não são igualmente importantes: se algumas toleram a remoção de parâmetros muito mais do que outras, podar mais as camadas tolerantes e menos as sensíveis deve produzir, para um mesmo grau de compressão, perda menor de desempenho preditivo do que as formas de distribuição adotadas pelas estratégias sem otimização (Seção 4.2), que aplicam a mesma taxa a todas as camadas ou decidem a distribuição apenas pela magnitude dos pesos. Para testar essa hipótese, fixa-se o **orçamento de poda** (a fração do total de parâmetros podáveis a ser removida do modelo, como 30% ou 50%) e varia-se apenas o modo como essa fração é distribuída entre as camadas (Seção 2.6.3).

A técnica de otimização foi escolhida, entre as alternativas consideradas, segundo três critérios:

- **O1 — Aderência à hipótese:** a técnica determina *quanto* podar em cada camada, que é o objeto da hipótese, e preserva o critério de importância da estratégia de poda, que continua a decidir *quais* parâmetros são removidos dentro de cada camada.
- **O2 — Ausência de um segundo modelo:** o modelo avaliado é o próprio modelo de referência após a poda, sem que outro modelo seja treinado para substituí-lo, como ocorre na *Knowledge Distillation*.
- **O3 — Compatibilidade com o regime *one-shot*:** a técnica se aplica ao modelo pré-treinado sem exigir treinamento durante a poda, como estabelece o critério P2 (Seção 4.2).

Os três critérios garantem que o modelo podado com otimização difira do modelo podado sem otimização em uma única variável: a distribuição do orçamento de poda entre as camadas. Mantidos o modelo, o critério de importância, o orçamento e a ausência de treinamento, qualquer diferença de desempenho entre os dois pode ser atribuída a essa distribuição.

4.3.1 Técnica Adotada: Análise de Sensibilidade por Camada

A técnica de otimização adotada neste trabalho, única entre as consideradas que atende aos três critérios, é a **análise de sensibilidade por camada**. Em vez de aplicar a mesma taxa de poda a todas as camadas, ela mede a importância de cada camada e distribui o orçamento de poda de acordo com essa medida: camadas mais tolerantes à remoção recebem taxas maiores, e camadas mais sensíveis, taxas menores.

A técnica é aplicada em duas fases. Na primeira, obtém-se o **perfil de sensibilidade** do modelo: poda-se uma camada de cada vez, mantendo todas as demais intactas, e mede-se o aumento da *perplexity* (Seção 4.1.3) em relação ao modelo de referência. Todas as camadas são podadas à mesma taxa fixa, denominada **taxa-sonda** — a fração da camada removida durante essa medição —, de modo que as degradações obtidas sejam comparáveis entre si; o perfil é o conjunto dessas degradações, uma por camada. Na segunda fase, o perfil é convertido em uma taxa de poda por camada, tanto maior quanto menor a degradação que a camada provocou, e as taxas são ajustadas em conjunto de modo que a fração total removida seja igual ao orçamento de poda pretendido.

Como cada estratégia de poda remove unidades de natureza distinta — pesos individuais ou estruturas inteiras —, o perfil é levantado separadamente para cada uma delas. A regra de alocação, o valor da taxa-sonda e os demais parâmetros adotados são apresentados no Capítulo 5 (Seção 5.4).

A comparação entre os modelos obtidos com e sem otimização, sob o mesmo orçamento de poda, visa identificar em que medida a alocação guiada por sensibilidade melhora o balanço entre desempenho preditivo e eficiência computacional em relação às formas de distribuição sem otimização.

4.4 Etapa 4 — *Fine-tuning* de Recuperação

As Etapas 2 e 3 operam em regime *one-shot*: o modelo é podado e avaliado imediatamente, sem nenhum treinamento após a poda. Esse regime é deliberado, pois mede o dano causado pela poda sem que nenhuma recuperação posterior o atenue. Ele difere, porém, do procedimento original das estratégias clássicas de poda, que retreinam o modelo depois de podá-lo, para que os parâmetros remanescentes compensem a ausência dos removidos (HAN et al., 2015; LI et al., 2017) (Seção 2.6). Esta etapa incorpora esse retreinamento ao experimento, com o objetivo de medir quanto da degradação causada pela poda pode ser recuperada.

O *fine-tuning* de recuperação consiste em continuar o treinamento do modelo já podado. Os parâmetros remanescentes são ajustados sobre a partição de treino da base de dados de avaliação, com o mesmo objetivo do pré-treinamento — a predição do próximo

token —, por um número fixo de passos de otimização, igual para todas as configurações retreinadas. Ao final, o modelo é avaliado novamente na partição de teste, pelas mesmas métricas da Seção 4.1.3. A recuperação é medida pela comparação entre a *perplexity* do modelo podado antes e depois do retreinamento.

5 Desenvolvimento

Este capítulo descreve as decisões concretas que decorrem dos critérios estabelecidos no Capítulo 4 e o experimento tal como foi efetivamente implementado. Após uma visão geral (Seção 5.1), o capítulo segue as mesmas quatro etapas da metodologia, na mesma ordem: Etapa 1 — Modelo de Referência (Seção 5.2); Etapa 2 — Poda sem Otimização (Seção 5.3); Etapa 3 — Poda com Otimização (Seção 5.4); e Etapa 4 — *Fine-tuning* de Recuperação (Seção 5.5). As seções finais tratam de aspectos comuns a todas as etapas: o *framework* e as bibliotecas empregadas, a organização do *pipeline* experimental, o ambiente de execução e as limitações reconhecidas.

5.1 Visão Geral

A pergunta que orienta o trabalho é empírica e comparativa. Não se pretende demonstrar que a poda comprime modelos — resultado já consolidado pela literatura (HAN et al., 2015; LI et al., 2017; BLALOCK et al., 2020) —, mas responder a duas questões mais específicas, sob condições controladas e reprodutíveis:

(A) Em que medida a redução *teórica* de parâmetros se converte em redução *real* de tempo de inferência e de consumo de energia, e como essa conversão depende da granularidade da poda?

(B) Uma alocação não uniforme do orçamento de poda, guiada por uma análise de sensibilidade por camada, melhora o compromisso entre compressão e qualidade em relação às alocações ingênuas?

Responder a essas duas perguntas exige que modelos comprimidos por estratégias distintas sejam avaliados exatamente pelo mesmo conjunto de métricas, na mesma base de dados e no mesmo *hardware*. Essa simetria de avaliação é condição necessária para que qualquer comparação seja interpretável, e é o princípio que organiza todo o desenvolvimento descrito adiante.

5.1.1 Discussão das Decisões Metodológicas

Todo desenho experimental implica escolhas que condicionam o escopo dos resultados. A Tabela 1 sintetiza as principais decisões adotadas, as alternativas consideradas e a justificativa de cada uma. As seções seguintes desenvolvem em maior profundidade as que são mais críticas para a interpretação dos resultados.

Tabela 1 – Síntese das principais decisões metodológicas do trabalho.

Decisão	**Alternativas consideradas**	**Escolha**	**Justificativa**
Base de dados	WikiText-103, Penn Treebank, OpenWebText, The Pile	**WikiText-2**	Atende aos quatro critérios da Seção 4.1.2; entre os que atendem, tem o menor conjunto de treino (Tabela 4).
Modelo	GPT-2 *medium*/*large*/XL, DistilGPT2, OPT-125M, Pythia-160M, BERT, Llama 2	**GPT-2 *small***	Atende aos seis critérios da Seção 4.1.1 e forma com o WikiText-2 um par que atende a D1–D4; entre os que atendem, é o de menor número de parâmetros (Tabela 3).
Estratégias de poda	SparseGPT, Wanda, *movement pruning*, poda de cabeças por gradiente	**Duas** (magnitude e estruturada)	Únicas alternativas que atendem a P1 e P2; juntas, atendem a P3 (Tabela 6). A restrição de tempo experimental inviabilizaria uma terceira com a mesma profundidade.
Regime de poda	Iterativo (podar e retreinar em ciclos)	***One-shot* primeiro**	Isola o efeito da poda do efeito do retreinamento; o *fine-tuning* entra depois, como etapa separada e mensurável.
Estratégia otimizada	Taxas dinâmicas, variação do momento da poda, destilação de conhecimento, quantização	**Sensibilidade por camada**	Única que atende a O1–O3 (Seção 4.3): altera apenas a distribuição do orçamento de poda entre as camadas, objeto da hipótese do trabalho.
Métrica principal	*Accuracy* em tarefas descendentes	**Perplexity**	Métrica intrínseca padrão para modelos de linguagem, mensurável sem *fine-tuning* em tarefa específica.
Ambiente	Máquina local, Google Colab	**Kaggle (GPU T4)**	Acesso gratuito a GPU dedicada com sessões longas o bastante para as varreduras completas.

Fonte: Elaboração própria.

5.2 Etapa 1 — Modelo de Referência

Na primeira etapa, o GPT-2 *small* (RADFORD et al., 2019) foi selecionado como modelo de referência. Utilizou-se a versão pré-treinada disponibilizada na plataforma Hugging Face, executada sem nenhuma modificação e avaliada no conjunto de teste da base de dados WikiText-2 (MERITY et al., 2017). Registraram-se a *perplexity*, a contagem de parâmetros, o tamanho em memória, os FLOPs, o tempo de inferência e o consumo de energia: as três primeiras métricas foram obtidas com a biblioteca PyTorch; os FLOPs, com a biblioteca `thop`; o tempo de inferência, com o temporizador `perf_counter` da linguagem Python; e o consumo de energia, com a biblioteca CodeCarbon (Tabela 5). O modelo de referência obteve *perplexity* de 25,17 (Tabela 2), valor contra o qual são comparadas todas as configurações podadas das etapas seguintes. As subseções a seguir apresentam a seleção do modelo (Seção 5.2.1), a seleção da base de dados (Seção 5.2.2) e as métricas registradas (Seção 5.2.3).

5.2.1 Seleção do Modelo

5.2.1.1 O GPT-2 *small*

O modelo de referência é o GPT-2 *small*, proposto por Radford et al. (2019) e descrito na Seção 2.3.5. Trata-se de um modelo autorregressivo composto exclusivamente pelo componente decodificador da arquitetura *Transformer*. Os pesos pré-treinados foram obtidos pela plataforma Hugging Face, que os disponibiliza publicamente, e carregados por meio da biblioteca PyTorch (PASZKE et al., 2019).

A Tabela 2 resume a configuração da variante utilizada.

Tabela 2 – Configuração do GPT-2 *small* utilizado como modelo de referência.

Característica	**Valor**
Blocos *Transformer* empilhados	12
Cabeças de atenção por bloco	12
Dimensão do modelo (d_{model})	768
Dimensão intermediária do MLP (d_{ff})	3.072
Tamanho do vocabulário	50.257
Janela de contexto máxima	1.024 *tokens*
Total de parâmetros	124.439.808
Tamanho em memória	474,70 MB
FLOPs por janela de 512 *tokens*	126,6 G
Perplexity no WikiText-2 (teste)	25,17

Fonte: Elaboração própria.

5.2.1.2 Justificativa da Escolha

A Seção 4.1.1 estabeleceu seis critérios objetivos (C1 a C6) para a seleção do modelo de referência. A Tabela 3 aplica esses critérios a nove candidatos: as quatro variantes do GPT-2 (RADFORD et al., 2019), o DistilGPT2 (SANH et al., 2019), o OPT-125M (ZHANG et al., 2022), o Pythia-160M (BIDERMAN et al., 2023), o BERT-base (DEVLIN et al., 2019) e o Llama 2 7B (TOUVRON et al., 2023). Os candidatos compreendem as quatro variantes da família GPT-2; três modelos abertos de porte próximo ao do GPT-2 *small*; e dois modelos de classes distintas, incluídos para verificar os limites dos critérios: um codificador e um modelo com bilhões de parâmetros. Como a seleção é conjunta (Seção 4.1.1), a tabela verifica também o critério D4 de cada candidato em relação ao WikiText-2.

Avaliar a inclusão de modelos pequenos posteriores a 2023 (por exemplo, SmolLM2, Qwen2.5-0.5B, Llama 3.2 1B), que a banca pode questionar.

Tabela 3 – Candidatos ao modelo de referência e verificação dos critérios de seleção.

Modelo	**Parâm.**	**Mem. FT (GB)**	**Downl. (mi)**	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**D4**
GPT-2 *small*	124M	2,0	930,1	✓	✓	✓	✓	✓	✓	✓
GPT-2 *medium*	355M	5,7	29,8	✓	✓	✓	✓	✓	✓	✓
GPT-2 *large*	774M	12,4	61,0	✓	✓	✓	✓	✓	✓	✓
GPT-2 XL	1,56B	24,9	17,0	✓	✓	×	✓	✓	✓	✓
DistilGPT2	82M	1,3	256,8	✓	✓	✓	×	—	✓	—
OPT-125M	125M	2,0	215,8	✓	✓	✓	✓	✓	✓	×
Pythia-160M	162M	2,6	27,2	✓	✓	✓	✓	—	✓	×
BERT-base	110M	1,8	3.246,2	×	✓	✓	✓	×	✓	×
Llama 2 7B	6,74B	107,8	34,6	✓	×	×	✓	✓	✓	—

C1: arquitetura decodificadora autorregressiva; C2: pesos abertos; C3: viabilidade do *fine-tuning*; C4: modelo original; C5: resultado de referência publicado; C6: implementação disponível (critérios definidos na Seção 4.1.1); D4: ausência de sobreposição entre o pré-treinamento e o WikiText-2 (Seção 4.1.2). —: não verificado, candidato já eliminado por outro critério. Mem. FT: memória mínima para o *fine-tuning* completo, a 16 bytes por parâmetro. Downl.: *downloads* acumulados na plataforma Hugging Face, em milhões.

Fonte: Elaboração própria, com dados de Radford et al. (2019), Rajbhandari et al. (2020), Frantar e Alistarh (2023), Sun et al. (2024), Gao et al. (2020) e da plataforma Hugging Face (acesso em 28 set. 2026).

As quatro variantes do GPT-2 atendem a C5 porque Radford et al. (2019) publicam, na Tabela 3 do artigo original, a *perplexity* de cada uma no WikiText-2: 29,41 (*small*), 22,76 (*medium*), 19,93 (*large*) e 18,34 (XL). A relação entre o valor publicado para o *small* e o medido neste trabalho, obtidos por protocolos distintos, é discutida na Seção 6.1.

O GPT-2 *small* acumula, além disso, 930,1 milhões de *downloads* na plataforma Hugging Face, o maior valor entre os candidatos de geração de texto considerados — o BERT-base tem mais *downloads*, mas é um codificador, já excluído por C1. Nos 30 dias anteriores ao acesso, o modelo somou 15,4 milhões de *downloads*, o segundo maior volume

entre todos os modelos de geração de texto da plataforma (acesso em 28 set. 2026). Esse dado reforça a representatividade do modelo, sem substituir os critérios de seleção.

Dos nove candidatos, seis não atendem a todos os critérios. O BERT-base cai em C1: sua arquitetura é apenas codificadora, pré-treinada com modelagem de linguagem mascarada (DEVLIN et al., 2019), de modo que a *perplexity* autorregressiva não se aplica — ela não é bem definida para modelos de linguagem mascarada (Hugging Face, 2026) —, o que o faz cair também em C5; e cai em D4, por ser pré-treinado sobre a Wikipédia em inglês (DEVLIN et al., 2019). O Llama 2 7B cai em C2: o acesso a seus pesos na plataforma Hugging Face está condicionado a aprovação manual (verificado em 28 set. 2026). Cai também em C3 — seus 6,74 bilhões de parâmetros exigiriam 107,8 GB para o *fine-tuning* completo, muito acima dos 16 GB disponíveis. Atende, porém, a C5: Sun et al. (2024) reportam *perplexity* de 5,12 no WikiText-2 para o modelo denso. O GPT-2 XL cai isoladamente em C3: Rajbhandari et al. (2020) apontam que um modelo de 1,5 bilhão de parâmetros exige, no mínimo, 24 GB de memória para o treinamento em precisão mista com Adam, acima do limite disponível. O DistilGPT2 cai em C4, por ser obtido por destilação de conhecimento supervisionada pelo GPT-2 de 124M (SANH et al., 2019).

O OPT-125M e o Pythia-160M exigem exame mais detido, pois atendem a C1–C4 e C6. O OPT-125M atende também a C5: Frantar e Alistarh (2023) reportam *perplexity* de 27,66 para o modelo denso no WikiText-2, em estudo de poda. Ambos, porém, caem em D4 em relação ao WikiText-2, pois incluem a Wikipédia no pré-treinamento: o OPT foi pré-treinado sobre uma seleção de componentes do The Pile que inclui a Wikipédia (ZHANG et al., 2022), e o Pythia sobre o The Pile completo (BIDERMAN et al., 2023), cujo componente Wikipedia (en) soma 6,38 GiB (GAO et al., 2020). O GPT-2, ao contrário, teve os documentos da Wikipédia removidos de sua base de dados de pré-treinamento (RADFORD et al., 2019) (Seção 5.2.2.2). Nenhum dos dois modelos forma par admissível com as demais bases de dados candidatas (Tabela 4): o WikiText-103 cai em D4 pelo mesmo motivo; o Penn Treebank e o OpenWebText caem em critérios que independem do modelo (D2 e D3, respectivamente); e The Pile cai em D4, por ser a base de dados de pré-treinamento do Pythia e fornecer parte do pré-treinamento do OPT.

Restam três candidatos que atendem a todos os critérios: as variantes *small*, *medium* e *large* do GPT-2, esta última com margem estreita em C3 — dos 12,4 GB estimados, restam cerca de 3,6 GB para as ativações. Aplicou-se, então, a regra de desempate da Seção 4.1.1: entre os candidatos que atendem a todos os critérios, adota-se o de menor número de parâmetros. O GPT-2 *medium* e o *large* têm, respectivamente, cerca de 2,9 e 6,2 vezes mais parâmetros que o *small*; a diferença é relevante porque a campanha experimental deste trabalho registrou 195 execuções completas de avaliação — 20 da poda por magnitude, 60 da poda estruturada, 48 do perfil de sensibilidade nas duas taxas-sonda, 60 da alocação por sensibilidade e 7 de *fine-tuning* de recuperação —, com

resultados reunidos em `codigo/resultados/`.

No ambiente adotado, uma época de *fine-tuning* do GPT-2 *small* sobre o conjunto de treino do WikiText-2 (4.743 blocos de 512 *tokens*, isto é, 2.428.416 *tokens*) levou 274 s, cerca de 4,6 minutos, o que corresponde a aproximadamente 8,9 mil *tokens* por segundo; os sete *fine-tunings* da campanha somaram cerca de 31 minutos. Supondo custo proporcional ao número de parâmetros — aproximação coerente com a contagem de FLOPs da Tabela 2, em que 126,6 GFLOPs por janela de 512 *tokens* correspondem a cerca de 2 operações por parâmetro por *token* —, o mesmo conjunto de *fine-tunings* levaria cerca de 1,5 hora com o *medium* e 3,2 horas com o *large*; trata-se de estimativas, não de medições diretas. A escolha recaiu, portanto, sobre o GPT-2 *small*.

Cabe observar que, embora seja a menor variante da família GPT-2, o modelo é cerca de cinco vezes maior que a ResNet-50, arquitetura tradicionalmente usada em estudos de poda em visão computacional. Essa diferença de escala é justamente o que torna o trabalho pertinente: as conclusões da literatura de poda foram estabelecidas majoritariamente em redes convolucionais, e não é evidente que se transfiram para modelos com esse perfil de parâmetros.

5.2.2 Seleção da Base de Dados

5.2.2.1 O WikiText-2

O WikiText-2 é uma base de dados de língua inglesa extraída de artigos verificados da Wikipédia, proposta por Merity et al. (2017), com *perplexity* do GPT-2 nessa base de dados reportada por Radford et al. (2019). Diferentemente de bases de dados construídas a partir de frases isoladas, o WikiText preserva a estrutura original dos artigos — parágrafos completos, pontuação e capitalização mantidas —, o que o torna adequado para medir a capacidade de um modelo de acompanhar contexto ao longo de textos extensos.

Empregou-se especificamente a variante `wikitext-2-raw-v1`, na qual o texto não sofre substituição de palavras raras por um marcador genérico. Essa escolha é necessária porque o GPT-2 utiliza tokenização por *subwords*, capaz de representar palavras fora do vocabulário decompondo-as em fragmentos; a variante processada anularia essa capacidade e distorceria a perplexity medida.

5.2.2.2 Justificativa da Escolha

A Seção 4.1.2 estabeleceu quatro critérios objetivos (D1 a D4) para a seleção da base de dados de avaliação. A Tabela 4 aplica esses critérios a cinco candidatos: o WikiText-2 e o WikiText-103 (MERITY et al., 2017), o Penn Treebank (MERITY et al., 2017), o OpenWebText (GOKASLAN; COHEN, 2019) e The Pile (GAO et al., 2020). Como D1 e D4 dependem do modelo, a tabela os verifica para a família GPT-2, adotada

na Seção 5.2.1; a verificação recíproca, para os demais candidatos a modelo, consta na Tabela 3.

Tabela 4 – Candidatas a base de dados de avaliação e verificação dos critérios de seleção.

Base de dados	**Treino**	**D1**	**D2**	**D3**	**D4**
WikiText-2	2,1 mi *tokens*	✓	✓	✓	✓
WikiText-103	103,2 mi *tokens*	✓	✓	✓	✓
Penn Treebank	0,93 mi *tokens*	✓	×	✓	✓
OpenWebText	39,8 GB	—	✓	×	×
The Pile	825 GiB	✓	✓	✓	×

D1: resultado de referência; D2: texto sem pré-processamento destrutivo; D3: divisão oficial; D4: ausência de sobreposição com o pré-treinamento (critérios definidos na Seção 4.1.2). D1 e D4 verificados para a família GPT-2. —: não verificado, candidato já eliminado por outro critério.

Fonte: Elaboração própria, com dados de Merity et al. (2017), Radford et al. (2019), Gao et al. (2020) e da plataforma Hugging Face (acesso em 28 set. 2026).

Dos cinco candidatos, três não atendem a todos os critérios. O Penn Treebank cai em D2: na versão usual, as palavras estão em caixa baixa, a pontuação foi removida e o vocabulário foi limitado a 10 mil palavras (MERITY et al., 2017), ao passo que o WikiText mantém pontuação, caixa original e números (MERITY et al., 2017), e a variante *raw* adotada neste trabalho não substitui palavras raras. O OpenWebText cai em D3, pois é distribuído com uma única partição, de treino, sem divisão oficial em validação e teste (8.013.769 documentos, 39,8 GB, verificado na plataforma Hugging Face); e em D4, por ser uma reprodução aberta do WebText, a base de dados de pré-treinamento do GPT-2 (GOKASLAN; COHEN, 2019). The Pile atende a D1 — Gao et al. (2020) avaliam as quatro variantes do GPT-2 em seu conjunto de teste, com a *perplexity* convertida em bits por *byte* — e a D3, por dispor de partições oficiais de treino, validação e teste — validação e teste com cerca de 0,1% dos dados cada, mais de 1 GiB cada (GAO et al., 2020); cai em D4 porque inclui o OpenWebText2 (62,77 GiB), construído pelo mesmo método do WebText, com votos no Reddit como filtro de qualidade dos links (GAO et al., 2020). Em sentido oposto, os autores do GPT-2 removeram do WebText todos os documentos da Wikipédia justamente para evitar sobreposição com conjuntos de avaliação (RADFORD et al., 2019); a sobreposição de 8-gramas medida por eles entre os conjuntos de teste e o treino do WebText é de 1,63% no WikiText-2, 2,42% no WikiText-103 e 0,88% no Penn Treebank (Tabela 6 de Radford et al. (2019)).

Restam dois candidatos que atendem a todos os critérios: o WikiText-2 e o WikiText-103. Ambos compartilham os mesmos conjuntos de validação e teste, com 217.646 e 245.569 *tokens*, respectivamente (MERITY et al., 2017); diferem apenas no conjunto de treino, cerca de 49 vezes maior no WikiText-103. Como o conjunto de teste é o mesmo, a avaliação de *perplexity* é idêntica nos dois; a diferença está inteiramente no custo do *fine-tuning*, determinado pelo treino. Com a vazão medida na Seção 5.2.1.2 — uma época sobre o treino do WikiText-2 em cerca de 4,6 minutos —, uma época sobre o

WikiText-103 levaria cerca de 3,8 horas por *fine-tuning*, e os sete *fine-tunings* da campanha passariam de cerca de 31 minutos para cerca de 25 horas (estimativa proporcional ao número de *tokens* de treino). Aplicou-se, então, a regra de desempate da Seção 4.1.2: adotou-se o de menor conjunto de treino, o WikiText-2.

Registre-se que a base de dados é parametrizável na implementação: a substituição pelo WikiText-103, de mesma natureza e cerca de cinquenta vezes maior, exige apenas a alteração de um argumento de linha de comando. Essa flexibilidade foi mantida deliberadamente para que trabalhos futuros possam verificar a generalidade dos resultados sem reescrever o código.

5.2.2.3 Pré-processamento e Tokenização

O pré-processamento foi intencionalmente mínimo, para que a perplexity medida refletisse a base de dados e não decisões de limpeza. As linhas do conjunto foram unidas por quebra dupla, preservando a separação entre parágrafos tal como na base de dados original, e o texto resultante foi tokenizado como um fluxo contínuo único.

A tokenização empregou o tokenizador original do GPT-2, baseado no algoritmo BPE (*Byte Pair Encoding*), com vocabulário de 50.257 *tokens*. Manter o tokenizador original é indispensável: os *embeddings* do modelo foram aprendidos para esse vocabulário específico, e qualquer substituição invalidaria os pesos pré-treinados.

5.2.2.4 Divisão dos Dados

Adotou-se a divisão oficial da base de dados. O **conjunto de teste** foi utilizado em todas as medições de perplexity reportadas, tanto do modelo denso quanto de todas as configurações comprimidas. O **conjunto de treinamento** foi utilizado exclusivamente na etapa de *fine-tuning* de recuperação (Seção 5.5). O **conjunto de validação** foi empregado apenas para acompanhar a evolução do treinamento ao longo dos passos, quando essa curva foi solicitada, e nunca para selecionar resultados reportados.

Essa separação estrita importa: como as varreduras *one-shot* não realizaram treinamento algum, elas não tiveram contato com o conjunto de treinamento, e a perplexity de teste é uma medida limpa do dano causado pela poda.

5.2.3 Métricas de Avaliação

Toda configuração — o modelo denso, as podadas pelos métodos da literatura e as podadas pela estratégia otimizada — foi avaliada pelo mesmo conjunto de métricas, cobrindo as três dimensões definidas na Seção 4.1.3. A Tabela 5 as resume.

Tabela 5 – Métricas de avaliação coletadas para cada configuração.

Métrica	**O que mede**	**Unidade**	**Ferramenta**
Perplexity	Qualidade da modelagem da linguagem; quanto menor, melhor	—	PyTorch
Parâmetros totais	Parâmetros existentes na arquitetura	unidades	PyTorch
Parâmetros não nulos	Parâmetros efetivamente diferentes de zero	unidades	PyTorch
Esparsidade real	Fração de parâmetros podáveis eliminados	%	PyTorch
Tamanho do modelo	Memória ocupada pelos parâmetros	MB	PyTorch
FLOPs	Operações de ponto flutuante por inferência	FLOPs	`thop`
Tempo de inferência	Tempo médio de uma passagem direta	ms	`perf_counter`
Energia consumida	Energia gasta durante a avaliação	kWh	`CodeCarbon`
Emissões de CO_2	Emissões equivalentes ao consumo	kg	`CodeCarbon`

Fonte: Elaboração própria.

5.2.3.1 Perplexity

A perplexity é a métrica de desempenho preditivo adotada. Ela foi calculada pelo método da janela deslizante, padrão para modelos autorregressivos: o fluxo de *tokens* do conjunto de teste foi percorrido em janelas do tamanho máximo de contexto do modelo, com passo menor que a janela, de modo que cada *token* fosse previsto dispondo de contexto anterior substancial. Apenas os *tokens* ainda não avaliados contribuíram para a perda em cada janela; os demais entraram como contexto.

Empregou-se janela de 1.024 *tokens* com passo de 512. Passos menores produzem estimativas mais precisas ao custo de mais computação; o valor adotado é o compromisso usual na literatura, e foi mantido idêntico em todas as configurações avaliadas.

5.2.3.2 Custo Computacional

A contagem de **parâmetros** é reportada em duas formas complementares. O total reflete o que existe na arquitetura; a contagem de não nulos reflete o que sobrou depois da poda. Na poda não estruturada as duas divergem — há zeros, mas a arquitetura é a mesma —, enquanto na poda estruturada elas coincidem, pois o que foi removido deixou de existir. Essa divergência é, por si só, um resultado: ela evidencia a diferença entre compressão contábil e compressão física.

A **esparsidade real** reportada nas tabelas foi sempre calculada sobre o conjunto podável, mas a natureza do cálculo acompanha a estratégia: na poda por magnitude é a fração de pesos anulados nas 48 matrizes; na poda estruturada é a fração de parâmetros

que deixaram de existir nas mesmas projeções, vieses incluídos, já que a remoção de uma estrutura elimina também o viés que lhe corresponde. As duas grandezas medem a mesma coisa — quanto do conjunto podável foi eliminado —, mas não são idênticas em denominador, e diferenças na segunda casa decimal entre estratégias não devem ser interpretadas.

Os **FLOPs** foram contabilizados por análise do grafo computacional. Registre-se uma particularidade da implementação: as projeções internas do GPT-2 não são camadas lineares convencionais, e sim um tipo específico de camada que a ferramenta de contagem não reconhece por padrão. Sem tratamento, a contagem ignoraria justamente as matrizes que a poda modifica, e a métrica sairia constante. Foi necessário, portanto, registrar uma regra de contagem própria para esse tipo de camada. Reconhece-se uma limitação remanescente: as multiplicações de matriz internas ao mecanismo de atenção não foram contabilizadas pela ferramenta em nenhuma configuração. Como a omissão é idêntica para todas as estratégias, as comparações permanecem válidas. A implementação prevê ainda um estimador analítico de reserva — duas operações por parâmetro não pertencente aos *embeddings*, por *token* processado —, acionado apenas se a contagem pelo grafo falhar. Todos os valores reportados no Capítulo 6 provêm da contagem pelo grafo, o que se verifica pelo fato de os FLOPs variarem com a poda estruturada; o estimador de reserva produziria uma grandeza de natureza diferente e não deve ser misturado a eles.

O **tempo de inferência** foi medido sobre uma entrada representativa de 512 *tokens*, com execuções de aquecimento descartadas — para mitigar efeitos de inicialização da GPU — seguidas de execuções cronometradas, das quais se reporta média e desvio-padrão.

5.2.3.3 Impacto Ambiental

O consumo de **energia** foi medido com o `CodeCarbon`, que monitora o *hardware* durante a execução do bloco de avaliação e reporta a energia consumida em quilowatts-hora. As **emissões de** CO_2 foram estimadas a partir desse consumo e da intensidade de carbono da rede elétrica onde o código executa.

Sobre esta última métrica cabe uma ressalva importante, identificada durante os experimentos: a intensidade de carbono utilizada variou conforme a sessão de execução. Em consequência, as emissões não são comparáveis entre execuções realizadas em sessões distintas, e a análise dos resultados apoia-se no consumo de energia, não nas emissões derivadas dele.

5.3 Etapa 2 — Poda sem Otimização

Na segunda etapa, aplicaram-se duas estratégias clássicas de poda: a poda não estruturada por magnitude (HAN et al., 2015) e a poda estruturada por norma L_1 (LI et al., 2017), esta última adaptada ao *Transformer*. Cada estratégia foi aplicada de forma independente e sempre a partir do modelo de referência original, em toda a faixa de esparsidade — de 10% a 90%, em passos de 10 pontos percentuais — e sob dois escopos de alocação distintos. A escolha dessas duas estratégias é deliberada: elas ocupam os dois extremos do espectro de granularidade e, como se verá, comportam-se de forma oposta quanto ao impacto real sobre o *hardware*.

5.3.1 Seleção das Estratégias

A Seção 4.2 estabeleceu três critérios (P1 a P3) para a escolha das estratégias de poda sem otimização. A Tabela 6 aplica esses critérios à poda por magnitude (HAN et al., 2015), à poda estruturada por norma L_1 (LI et al., 2017) e a quatro alternativas da literatura.

Tabela 6 – Estratégias de poda candidatas e verificação dos critérios de seleção.

Estratégia	**Granularidade**	**Critério de importância**	**Ano**	**Citações**	**P1**	**P2**
Magnitude	fina	$\|w\|$ de cada peso	2015	8.047	✓	✓
Norma L_1	grossa	norma L_1 dos pesos da unidade	2017	4.214	✓	✓
SparseGPT	fina	reconstrução com dados de calibração	2023	1.552	×	✓
Wanda	fina	$\|w\|$ × norma das ativações	2024	999	×	✓
Movement pruning	fina	movimento dos pesos no *fine-tuning*	2020	654	×	×
Cabeças por gradiente	grossa	gradiente da perda em relação à máscara	2019	1.513	×	✓

P1: importância calculada apenas a partir dos pesos; P2: aplicação *one-shot* (critérios definidos na Seção 4.2). P3 refere-se ao conjunto de estratégias e é discutido no texto. Ano: ano de publicação do artigo que propôs a estratégia. Citações: citações desse artigo na base Semantic Scholar, indicador complementar de representatividade, que não integra os critérios.

Fonte: Elaboração própria, com base em Han et al. (2015), Li et al. (2017), Frantar e Alistarh (2023), Sun et al. (2024), Sanh, Wolf e Rush (2020) e Michel, Levy e Neubig (2019), e com dados da base Semantic Scholar (acesso em 3 out. 2026).

As quatro alternativas caem em P1, pois todas estimam a importância com base em dados. O SparseGPT e o Wanda, referências recentes da poda não estruturada de LLMs, utilizam 128 sequências de calibração extraídas da base de dados C4: o primeiro para reconstruir, com informação de segunda ordem, os pesos remanescentes de cada camada (FRANTAR; ALISTARH, 2023); o segundo para ponderar a magnitude de cada peso pela norma da ativação de entrada correspondente (SUN et al., 2024). A importância das cabeças em Michel, Levy e Neubig (2019) é estimada pelo gradiente da perda sobre uma base de dados. A *movement pruning* cai também em P2, pois a importância de cada peso é aprendida ao longo do *fine-tuning* (SANH; WOLF; RUSH, 2020). A exclusão dessas estratégias não decorre de desempenho inferior: elas já incorporam o tipo de informação

que a análise de sensibilidade introduz, e sua combinação com a alocação por sensibilidade fica fora do escopo deste trabalho. Restam a poda por magnitude e a poda estruturada por norma L_1, que atendem a P1 e P2 e, juntas, atendem a P3: a primeira opera sobre pesos individuais e a segunda sobre unidades arquiteturais inteiras.

Quanto ao indicador complementar de representatividade, os artigos que propuseram as duas estratégias adotadas são os mais citados entre as seis candidatas: 8.047 citações para Han et al. (2015) e 4.214 para Li et al. (2017), contra 1.552 do SparseGPT, o terceiro (Semantic Scholar, acesso em 3 out. 2026). Como as citações se acumulam com o tempo, a diferença reflete em parte a antiguidade dos trabalhos — as quatro alternativas foram publicadas entre 2019 e 2024. O dado reforça a condição de referência das duas estratégias na literatura de poda, sem substituir os critérios de seleção. A mesma condição é corroborada pela meta-análise de Blalock et al. (2020), que reuniu os resultados de 81 trabalhos de poda e aponta como um dos resultados consistentes da área que a poda pela magnitude dos pesos comprime substancialmente as redes sem reduzir a acurácia.

5.3.2 Parâmetros Podáveis

Uma decisão de implementação que condiciona todos os resultados é a definição de quais parâmetros são elegíveis à poda. Adotaram-se, por convenção consolidada na literatura, apenas as matrizes de peso das projeções lineares internas aos blocos *Transformer*, a saber, quatro por bloco:

- `attn.c_attn` — projeção que produz consulta, chave e valor;
- `attn.c_proj` — projeção de saída da atenção;
- `mlp.c_fc` — primeira transformação linear do MLP;
- `mlp.c_proj` — segunda transformação linear do MLP.

Com 12 blocos, são 48 matrizes podáveis. Ficaram deliberadamente fora do escopo da poda:

- as matrizes de *embedding* de *token* (`wte`) e de posição (`wpe`), que codificam o vocabulário e não representam computação repetida a cada camada;
- a cabeça de linguagem (`lm_head`), que no GPT-2 compartilha pesos com `wte` — podá-la corromperia os *embeddings*;
- vieses e parâmetros de normalização de camada, que são numericamente baratos e desproporcionalmente sensíveis.

Essa restrição tem consequência direta na leitura dos resultados: quando se afirma que uma configuração tem 50% de esparsidade, trata-se de 50% das matrizes podáveis, não do modelo inteiro. Como os *embeddings* do GPT-2 concentram parcela expressiva dos parâmetros, a esparsidade sobre o total é sempre menor que a esparsidade sobre o conjunto podável, e as duas grandezas são reportadas separadamente.

5.3.3 Método 1 — Poda Não Estruturada por Magnitude

O primeiro método segue Han et al. (2015). O critério de importância é o valor absoluto do peso: assume-se que pesos próximos de zero contribuem pouco para a saída e podem ser anulados. Dada uma taxa de esparsidade alvo, calculou-se o limiar correspondente ao quantil apropriado da distribuição de $|w|$ e zeraram-se todos os pesos abaixo dele.

A implementação preserva a característica essencial do método: os pesos são **zerados**, não removidos. As matrizes conservam suas dimensões originais e a arquitetura permanece intacta — o que, como discutido na Seção 2.6, é precisamente a razão pela qual esse método não reduz o custo de inferência em *hardware* convencional.

5.3.4 Método 2 — Poda Estruturada por Norma L_1

O segundo método adapta Li et al. (2017) ao *Transformer*. O trabalho original poda filtros convolucionais ordenados por norma L_1; aqui, as unidades estruturais equivalentes são as **cabeças de atenção** e os **neurônios da camada intermediária do MLP**. Para cada uma dessas estruturas calculou-se a norma L_1 dos pesos que lhe correspondem, e removeram-se as de menor norma.

A diferença decisiva em relação ao método anterior é que a remoção é **física**. As matrizes foram efetivamente reconstruídas com dimensões menores: as colunas de consulta, chave e valor associadas às cabeças removidas foram eliminadas de `c_attn`, as linhas correspondentes saíram de `c_proj`, e a contabilidade interna do módulo de atenção — número de cabeças e dimensão de divisão — foi atualizada de acordo. O resultado não é um modelo esparso, mas uma arquitetura menor e densa.

A implementação admite três alvos: apenas cabeças, apenas neurônios MLP, ou ambos simultaneamente. Uma salvaguarda impede que qualquer bloco fique sem nenhuma cabeça ou sem nenhum neurônio, situação que romperia o fluxo residual e inviabilizaria o modelo.

Convém registrar que a esparsidade alvo, neste método, é expressa como fração de **estruturas** removidas, não de parâmetros. Duas razões fazem a fração de parâmetros efetivamente eliminada divergir da taxa solicitada, e por isso ela é reportada separadamente em todas as tabelas. A primeira é que cabeças e neurônios têm custos diferentes em

parâmetros. A segunda é a granularidade discreta das estruturas: o número de unidades a remover é a parte inteira do produto entre a taxa e a contagem disponível, o que nos neurônios do MLP — 3.072 por bloco — é praticamente imperceptível, mas nas cabeças de atenção, que são apenas 12 por bloco, produz arredondamentos visíveis. Uma taxa de 90%, por exemplo, remove 10 das 12 cabeças, ou seja, 83%.

5.3.5 Escopos de Alocação

Ambos os métodos foram avaliados sob dois escopos, que representam formas distintas de distribuir o orçamento de poda entre as camadas:

- **Global:** o critério de importância é aplicado ao modelo inteiro de uma só vez. Um único limiar — ou uma única ordenação de estruturas — decide o que sai, de modo que camadas com pesos sistematicamente menores cedem proporcionalmente mais. Na poda estruturada com os dois alvos simultâneos, o escopo global compreende duas ordenações independentes, uma de cabeças e outra de neurônios, cada uma abrangendo as doze camadas: as normas L_1 de uma cabeça de atenção e de um neurônio do MLP não são grandezas comparáveis, pois agregam números distintos de pesos, e ordená-las em conjunto faria a escolha depender do tamanho da estrutura em vez de sua importância relativa.
- **Uniforme:** cada camada recebe exatamente a mesma fração de poda, decidida isoladamente dentro dela. É a alocação ingênua por excelência, pois ignora qualquer diferença de importância entre camadas.

O contraste entre esses dois escopos é o que estabelece a linha de base contra a qual a estratégia otimizada é julgada. Adianta-se que nenhum deles domina o outro em toda a faixa de esparsidade, e é essa constatação que motiva a análise de sensibilidade.

5.3.6 Taxas de Poda Avaliadas

Cada combinação de método e escopo foi avaliada em dez níveis: 0% (o modelo denso, incluído como controle interno de cada varredura) e de 10% a 90% em passos de 10 pontos percentuais. A varredura completa da faixa foi uma escolha deliberada: como não se conhecia de antemão onde estaria o ponto de colapso de cada estratégia, restringir os níveis a uma faixa estreita poderia ocultar exatamente o comportamento que se desejava caracterizar.

Cada nível partiu de um modelo recarregado do zero, e não do modelo podado no nível anterior. Sem esse cuidado, a poda seria cumulativa e os níveis deixariam de ser independentes entre si.

5.4 Etapa 3 — Poda com Otimização

Na terceira etapa, aplicou-se a técnica de otimização selecionada na Seção 4.3 — a análise de sensibilidade por camada —, que constitui a contribuição central do trabalho. Ela responde a uma limitação comum aos métodos da Etapa 2: nenhum deles mede o quanto cada camada realmente importa. O escopo uniforme trata todas as camadas como intercambiáveis; o escopo global decide com base na magnitude dos pesos, que é um *proxy* de importância, não uma medida dela. A estratégia proposta substitui esse *proxy* por uma medição direta, em duas fases, e foi avaliada nos mesmos orçamentos de poda das varreduras da Etapa 2.

5.4.1 Fase 1 — Perfil de Sensibilidade

Para cada camada ℓ do modelo e para cada estratégia de poda, podou-se **exclusivamente** aquela camada a uma taxa-sonda fixa, mantendo todas as demais intactas, e mediu-se a perplexity resultante. A degradação foi registrada em escala logarítmica:

$$\Delta_\ell = \ln\left(\frac{\mathrm{PPL}_\ell}{\mathrm{PPL}_{\mathrm{base}}}\right), \tag{5.1}$$

onde PPL_ℓ é a perplexity do modelo com apenas a camada ℓ podada. O logaritmo é necessário porque a perplexity de modelos podados sem recuperação varia por ordens de grandeza; em escala linear, uma única camada catastrófica tornaria todas as demais indistinguíveis entre si.

Adotou-se taxa-sonda de 50%. O procedimento não envolveu treinamento e produziu, ao final, um perfil com uma medida de sensibilidade por camada, para cada estratégia. Como as duas estratégias removem coisas diferentes, os perfis foram levantados separadamente e não são intercambiáveis — ponto que a Seção 6.4.3 demonstra experimentalmente.

Como a escolha da taxa-sonda é arbitrária, o perfilamento completo foi replicado a uma segunda taxa — 30% —, nas duas estratégias, de modo a verificar se a ordenação de sensibilidade obtida é propriedade das camadas ou artefato do regime em que a sonda opera. A réplica cumpriu função de verificação e não alimentou nenhuma alocação: todas as taxas por camada reportadas derivam do perfil de 50%. O resultado da verificação é apresentado na Seção 6.4.5.

5.4.2 Fase 2 — Alocação Adaptativa de Taxas

O perfil foi então convertido em uma taxa de poda por camada. Atribuiu-se a cada camada uma taxa proporcional à sua robustez — o inverso da degradação medida — elevada a um expoente β e escalada por um fator único α, escolhido de modo que a

média das taxas, ponderada pelo número de parâmetros podáveis de cada camada, iguale o orçamento global s pretendido:

$$r_\ell = \min\Big(\alpha \cdot \Delta_\ell^{-\beta},\ r_{\max}\Big), \qquad \frac{\sum_\ell w_\ell\, r_\ell}{\sum_\ell w_\ell} = s, \tag{5.2}$$

em que w_ℓ é o número de parâmetros podáveis da camada ℓ e $r_{\max} = 0{,}95$ é um teto que impede que mesmo a camada mais robusta seja integralmente zerada. Como a fração podada é monótona em α, o fator foi obtido por bisseção.

A ponderação por w_ℓ não produz efeito no modelo aqui empregado: os doze blocos do GPT-2 são idênticos em estrutura e, portanto, têm exatamente o mesmo número de parâmetros podáveis, de modo que a média ponderada se reduz à média simples das taxas. Ela foi mantida na formulação por generalidade — em arquiteturas de blocos heterogêneos, ignorá-la faria o orçamento global ser sistematicamente descumprido.

O expoente β controla o quanto o perfil é levado a sério. Com $\beta = 1$ emprega-se o inverso puro da degradação; valores intermediários amortecem a razão entre camadas. Esse amortecimento não é um detalhe: a razão de robustez entre duas camadas é ilimitada, e sem controle uma única camada muito robusta absorveria praticamente todo o orçamento. Os experimentos avaliaram $\beta = 1{,}0$ e $\beta = 0{,}5$ sobre o mesmo perfil, o que forneceu a ablação da função de alocação sem exigir a repetição da fase 1, que é a etapa cara.

O valor $\beta = 0$ foi avaliado também, mas com outra finalidade: como o fator $\Delta_\ell^{-\beta}$ vale 1 para toda camada, todas recebem a mesma taxa e a regra deve reproduzir a poda uniforme. Serve, portanto, de controle de corretude da alocação, e não de alocação candidata. O controle é exato na poda estruturada; na poda por magnitude ele não se aplica, porque a poda por camada adota um limiar único por bloco — compartilhado pelas quatro projeções — enquanto a varredura uniforme decide um limiar por matriz. As duas alcançam a mesma esparsidade agregada, distribuída de forma distinta dentro do bloco. A verificação é reportada na Seção 6.5.3.

Registre-se, por fim, uma salvaguarda da implementação não visível na Equação 5.2: como a degradação medida pela sonda pode ser nula ou negativa — há casos em que remover cabeças de atenção *melhora* a perplexity, fenômeno reportado por Michel, Levy e Neubig (2019) —, a robustez Δ_ℓ^{-1} foi calculada sobre um piso positivo, de modo que tais camadas entrem na alocação como as mais robustas em vez de produzir divisão por zero. Nos perfis efetivamente levantados neste trabalho o piso não foi acionado: a menor degradação observada foi de 0,0346, duas ordens de grandeza acima dele.

5.5 Etapa 4 — *Fine-tuning* de Recuperação

As Etapas 2 e 3 operaram em regime *one-shot*, isto é, podaram e avaliaram sem retreinamento, de modo a isolar o dano causado pela poda. Esse regime, porém, não corresponde ao procedimento da literatura: tanto Han et al. (2015) quanto Li et al. (2017) retreinam o modelo após podá-lo. Na quarta etapa, esse retreinamento foi incorporado ao experimento: sete configurações foram submetidas ao *fine-tuning* de recuperação, e a *perplexity* de cada uma foi medida antes e depois do retreinamento, o que indica quanto da degradação é recuperável. As subseções a seguir apresentam as configurações retreinadas, o procedimento aplicado a cada uma, os cuidados que condicionam a validade da medição e os parâmetros do treinamento.

5.5.1 Configurações Retreinadas

O retreinamento foi aplicado às seguintes configurações das Etapas 2 e 3:

- **Controle denso:** o modelo de referência, sem poda, cuja função é descrita na Seção 5.5.3;
- **Poda não estruturada — poda por magnitude**, com escopo uniforme, a 30% e a 50% de esparsidade;
- **Poda estruturada — poda de cabeças de atenção e de neurônios do MLP**, com os dois alvos simultâneos e escopo global, a 10% e a 30% de estruturas removidas;
- **Poda estruturada com alocação por sensibilidade**, guiada pelo perfil da própria poda estruturada, com $\beta = 1$, nas mesmas duas taxas.

Em cada estratégia, retreinou-se o escopo de alocação ingênuo de menor *perplexity* na faixa em que a escolha de escopo tem consequência prática: o uniforme na poda por magnitude, que supera o global entre 30% e 70% (Seção 6.2), e o global na poda estruturada (Seção 6.3). A alocação por sensibilidade foi retreinada apenas na poda estruturada, a única estratégia que converteu a compressão em redução de tempo de inferência e de energia (Seção 6.3), com o expoente de melhor desempenho nessa estratégia (Seção 6.5.4). As duas taxas, 10% e 30%, são comuns à alocação global e à guiada, o que permite comparar as duas formas de distribuição também depois do retreinamento, e pertencem à faixa de 10% a 50% em que a alocação guiada superou a global em regime *one-shot* (Seção 6.5.2). Na poda por magnitude, 30% é o primeiro nível acima da região utilizável em regime *one-shot*, limitada a 20% (Seção 6.2), e 50% é um nível de degradação severa, com *perplexity one-shot* de 473,28.

5.5.2 Procedimento

Cada configuração passou pelo mesmo procedimento, implementado no módulo `finetune` (Tabela 8):

1. o modelo de referência foi recarregado a partir dos pesos pré-treinados e podado pelas mesmas funções das Etapas 2 e 3, de modo que o modelo retreinado fosse idêntico ao avaliado em regime *one-shot*;
2. mediu-se a *perplexity* do modelo podado, ponto de partida da recuperação; nas sete configurações, o valor obtido reproduziu, em todos os dígitos registrados, o das varreduras *one-shot*;
3. na poda não estruturada, capturou-se a máscara binária dos pesos remanescentes (Seção 5.5.3);
4. o modelo foi retreinado sobre a partição de treino do WikiText-2, com os parâmetros da Tabela 7;
5. o modelo retreinado foi avaliado na partição de teste por todas as métricas da Etapa 1 (Tabela 5), pelo mesmo procedimento aplicado aos modelos não retreinados.

O retreinamento atualizou todos os parâmetros do modelo podado — inclusive *embeddings*, vieses e parâmetros de normalização —, e não apenas as matrizes podáveis (Seção 5.3.2). O modelo avaliado é o obtido ao final do último passo do otimizador: a partição de validação não foi utilizada, e não houve parada antecipada nem seleção do melhor ponto do treinamento. Com isso, a quantidade de retreinamento é idêntica em todas as configurações, condição para que suas *perplexities* pós-retreinamento sejam comparáveis entre si.

5.5.3 Controle Denso e Preservação da Esparsidade

Duas decisões de implementação merecem registro por afetarem a validade dos resultados.

A primeira é o **controle denso**. O GPT-2 não foi pré-treinado no WikiText-2, de modo que o simples *fine-tuning* do modelo denso nessa base de dados já reduz a perplexity por adaptação de domínio, sem poda alguma envolvida. Sem essa linha de controle, seria impossível separar a recuperação do dano da poda da mera adaptação à base de dados. O modelo denso foi, portanto, retreinado pelo mesmo número de passos e avaliado junto às demais configurações.

A segunda é a **preservação da esparsidade**. Na poda não estruturada os pesos são apenas zerados, e o gradiente os restauraria já no primeiro passo do otimizador; ao

final do treinamento, o modelo estaria novamente denso. Como em Han et al. (2015), manteve-se uma máscara binária fixa, capturada logo após a poda e reaplicada depois de cada passo do otimizador. Registre-se que zerar o gradiente não seria suficiente: o otimizador utilizado possui momento e regularização por decaimento de pesos, e ambos alteram um parâmetro mesmo quando seu gradiente é nulo. A esparsidade medida ao final do treinamento foi registrada como verificação de integridade e coincidiu, nas duas configurações de magnitude, com a medida logo após a poda (Seção 6.6.4). Na poda estruturada não houve máscara, pois as estruturas já haviam sido fisicamente removidas.

5.5.4 Parâmetros do Treinamento

Para o treinamento, as linhas da partição de treino do WikiText-2 foram concatenadas em um único texto, tokenizado e dividido em blocos contíguos e disjuntos de 512 *tokens*; o resto final, menor que um bloco, foi descartado. Resultaram 4.743 blocos, ou 2.428.416 *tokens*. A divisão difere da adotada na avaliação, que percorre o texto por janelas deslizantes sobrepostas (Seção 5.2.3): na avaliação, a sobreposição garante contexto anterior a cada *token* avaliado; no treinamento, blocos disjuntos fazem de cada bloco uma amostra independente. A Tabela 7 reúne os parâmetros adotados, idênticos nas sete configurações.

Tabela 7 – Parâmetros do *fine-tuning* de recuperação.

Parâmetro	**Valor**
Blocos de treino (512 *tokens* cada)	4.743
Blocos por lote	4
Lotes acumulados por passo do otimizador	4
Lote efetivo por passo	16 blocos (8.192 *tokens*)
Passos do otimizador	297 (uma época)
Otimizador	AdamW
Taxa de aprendizado máxima	5×10^{-5}
Aquecimento linear da taxa de aprendizado	17 passos (6% do total, truncado)
Decaimento da taxa de aprendizado	linear, até zero no passo 297
Decaimento de pesos	0,01
Limite da norma do gradiente	1,0
Precisão numérica	mista (FP16 e FP32)
Semente aleatória	42

Fonte: Elaboração própria, com valores de `codigo/finetune.py` *e de* `codigo/resultados/finetune.csv`.

Os gradientes de quatro lotes consecutivos, de quatro blocos cada, foram acumulados antes de cada passo do otimizador, o que equivale a um lote efetivo de 16 blocos; a perda de cada lote foi dividida por quatro, de modo que o gradiente acumulado correspondesse à média dos lotes, e não à soma. O número de passos foi calculado para corresponder

a uma época, isto é, a uma passagem pelo conjunto de treino: os 4.743 blocos, divididos em lotes efetivos de 16, resultam em 297 passos, com arredondamento para cima.

O otimizador AdamW (LOSHCHILOV; HUTTER, 2019) aplica o decaimento de pesos separadamente da atualização pelo gradiente. O decaimento foi aplicado apenas às matrizes de peso, deixando de fora vieses e parâmetros de normalização, conforme prática consolidada. A taxa de aprendizado cresceu linearmente durante os 17 primeiros passos e decresceu linearmente até zero no último, e a norma do gradiente foi limitada a 1,0 antes de cada passo. A taxa de aprendizado e o lote efetivo estão entre os valores recomendados por Devlin et al. (2019) para o *fine-tuning* de *Transformers* pré-treinados — taxa de 2×10^{-5} a 5×10^{-5} e lotes de 16 ou 32 exemplos —; o decaimento de pesos de 0,01 e o aquecimento seguido de decaimento linear são os adotados por esses autores no pré-treinamento do BERT. Nenhum desses valores foi ajustado para este experimento (Seção 5.9). A quantidade de retreinamento, uma época, fica abaixo das duas a quatro épocas recomendadas pelos mesmos autores; o que essa limitação impede de concluir é discutido na Seção 6.6.4.

A passagem direta e a retropropagação executaram em **precisão mista** (MICIKEVICIUS et al., 2018): as operações foram realizadas em meia precisão (FP16), com os pesos mantidos em precisão simples (FP32) e escalonamento dinâmico da perda, que evita o subtransbordamento dos gradientes em meia precisão. A escolha aproveita as unidades de cálculo especializadas da GPU Tesla T4, que alcança 65 TFLOPS em precisão mista contra 8,1 TFLOPS em precisão simples (NVIDIA Corporation, 2019), e corresponde à configuração pressuposta na estimativa de memória do critério C3 (Seção 4.1.1). Ela introduz, porém, uma fonte adicional de variação numérica que não existe nas etapas *one-shot*, todas executadas em precisão simples. O consumo de energia do treinamento foi medido pela biblioteca CodeCarbon separadamente do consumo da avaliação: trata-se de um custo pago uma única vez, que precisa ser amortizado contra a economia por inferência para que o balanço energético do trabalho feche corretamente.

Diferentemente de todas as etapas anteriores, esta é estocástica — a ordem dos lotes e o *dropout* introduzem variabilidade — e, portanto, é a única em que a semente aleatória efetivamente altera o resultado. Cada configuração foi retreinada uma única vez, com a semente 42, reiniciada antes de cada configuração. A ordem dos lotes, sorteada por um gerador próprio iniciado com essa semente, foi a mesma nas sete configurações, de modo que as diferenças entre elas não decorrem da ordem em que os dados foram apresentados. A ausência de repetições com sementes distintas é discutida entre as limitações (Seção 5.9).

5.6 *Framework* de Implementação

A implementação foi feita em Python, com o *framework* PyTorch (PASZKE et al., 2019) para construção, manipulação e treinamento dos modelos. O PyTorch foi escolhido por seu sistema de diferenciação automática, que constrói dinamicamente o grafo computacional e simplifica a implementação do *backpropagation* durante o *fine-tuning*, e por seu suporte nativo à aceleração em GPU.

Os pesos pré-treinados foram obtidos na plataforma Hugging Face por meio da biblioteca `transformers` (WOLF et al., 2020), que implementa sobre o PyTorch as arquiteturas dos modelos candidatos. A Hugging Face não é uma biblioteca, mas um ambiente colaborativo de hospedagem e distribuição de modelos, que hospedava mais de 3,1 milhões de modelos em 28 set. 2026, entre eles diversas famílias de modelos de linguagem, como GPT-2, OPT, Pythia, Llama e BERT; a biblioteca que executa os modelos é o PyTorch.

As demais bibliotecas empregadas são:

- `datasets`: carregamento da base de dados WikiText a partir da plataforma Hugging Face;
- `thop`: contagem de operações de ponto flutuante por análise do grafo computacional;
- `CodeCarbon` (COURTY et al., 2023): medição empírica do consumo de energia e estimativa das emissões de CO_2, a partir do monitoramento do *hardware* combinado à intensidade de carbono da rede elétrica local.

Uma observação relevante sobre a implementação da poda: ambas as estratégias foram implementadas manualmente, sem recurso a bibliotecas dedicadas. No caso da poda estruturada, essa decisão foi imposta pelo ambiente — as versões mais recentes da biblioteca `transformers` removeram a interface de poda de cabeças que existia em versões anteriores. A remoção física das estruturas foi, portanto, realizada diretamente: as matrizes foram reconstruídas com as dimensões reduzidas e a contabilidade interna do módulo de atenção foi atualizada. A implementação da poda é coberta por uma suíte de testes automatizados (`test_poda`), executada sobre um GPT-2 de dimensões reduzidas e pesos aleatórios — o que está sob verificação é a mecânica da poda, não os pesos pré-treinados. O teste central é o de **equivalência numérica**: remover fisicamente um conjunto de cabeças de atenção, ou de neurônios do MLP, deve produzir exatamente as mesmas saídas que apenas anular os pesos correspondentes no modelo de dimensões originais. A equivalência é matematicamente necessária — anular as fatias de consulta, chave e valor de uma cabeça zera seu vetor de valores, de modo que sua contribuição à saída da atenção se anula qualquer que seja a distribuição de atenção calculada —, e por isso qualquer erro de indexação na reconstrução das matrizes a quebraria. Os demais

testes verificam que a contabilidade interna do módulo de atenção permanece coerente com as formas dos tensores, que a salvaguarda preserva ao menos uma estrutura por bloco mesmo sob taxa integral, que as estruturas removidas são efetivamente as de menor norma L_1, que a poda por magnitude atinge a esparsidade solicitada preservando as dimensões, que a regra de alocação respeita o orçamento e poda menos a camada mais sensível, e que a máscara de esparsidade sobrevive a um passo do otimizador.

5.7 *Pipeline* Experimental

O código está organizado em módulos de responsabilidade única, o que permite que cada estratégia de poda reutilize exatamente a mesma rotina de avaliação — condição para que as comparações sejam legítimas. A Tabela 8 descreve os módulos e a Figura 7 representa o fluxo experimental.

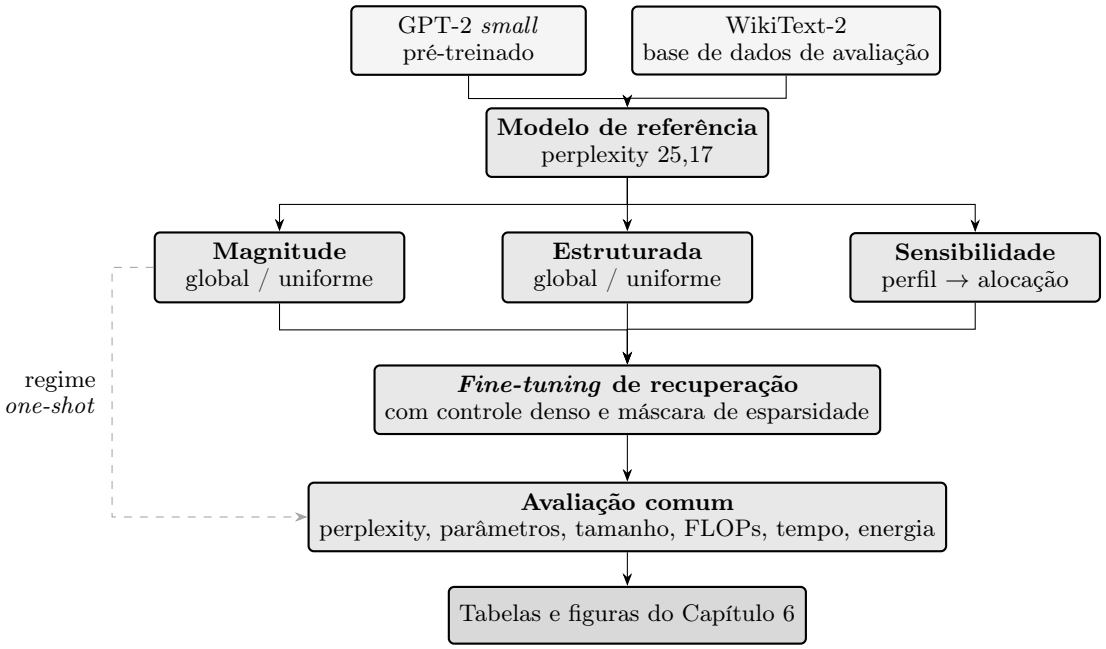
Tabela 8 – Organização modular da implementação.

Módulo	**Responsabilidade**
`config`	Configuração central: modelo, base de dados, parâmetros de avaliação, semente e ambiente.
`utils`	Fixação das sementes e escrita incremental dos resultados em arquivo.
`data`	Carregamento e tokenização da base de dados.
`model`	Carregamento do GPT-2 pré-treinado.
`eval`	Perplexity por janela deslizante e métricas de custo computacional.
`energy`	Medição de energia e emissões.
`prune_magnitude`	Poda não estruturada por magnitude, nos dois escopos.
`prune_structured`	Poda estruturada de cabeças e neurônios, com remoção física.
`sensitivity`	Perfil de sensibilidade (fase 1) e alocação adaptativa (fase 2).
`finetune`	*Fine-tuning* de recuperação, com preservação de esparsidade.
`run`	Orquestração da avaliação do modelo de referência.
`test_poda`	Testes de corretude da poda, descritos na Seção 5.6.

Fonte: Elaboração própria.

Como indica a linha tracejada da figura, as varreduras *one-shot* contornam a etapa de *fine-tuning* e vão direto à avaliação. Essa bifurcação é o que permite separar, nos resultados, o dano causado pela poda daquilo que o retreinamento consegue recuperar.

Figura 7 – Fluxo experimental do trabalho. Todas as estratégias convergem para a mesma rotina de avaliação, o que garante a simetria necessária às comparações.



Fonte: Elaborada pelo autor.

5.8 Ambiente de Execução e Reprodutibilidade

Os experimentos foram executados na plataforma Kaggle, em sessões com GPU NVIDIA Tesla T4. A implementação detecta automaticamente o dispositivo disponível, o que permite executar localmente em CPU para testes rápidos sem alteração de código.

Quanto à reprodutibilidade, três aspectos merecem registro.

O primeiro é que as varreduras *one-shot* são **determinísticas**. Não há treinamento, o modelo opera em modo de avaliação, o conjunto de teste é fixo e os critérios de poda são ordenações determinísticas sobre pesos fixos. Verificou-se empiricamente que execuções independentes, em sessões distintas, reproduziram a perplexity até a última casa decimal. Repetições com sementes diferentes, nessas etapas, produziriam linhas idênticas e não acrescentariam informação. A semente passa a importar apenas no *fine-tuning*.

O segundo é que as medições de **tempo e energia dependem da sessão**. Observou-se que o mesmo modelo denso, sem qualquer poda, registrou tempos e consumos sensivelmente distintos em sessões diferentes da plataforma, o que se atribui à variação de *hardware* físico e de carga concorrente. Por essa razão, comparações de custo entre varreduras executadas em sessões distintas foram sempre feitas em termos relativos ao modelo denso da própria sessão, nunca em valores absolutos.

O terceiro é que cada configuração experimental gravou uma linha em arquivo de resultados, contendo não apenas as métricas mas também os parâmetros que a gera-

ram — estratégia, escopo, taxa alvo, taxa efetiva e, no caso da estratégia otimizada, as taxas atribuídas a cada camada. Isso permite rastrear qualquer número apresentado no Capítulo 6 até a configuração exata que o produziu.

5.9 Limitações Metodológicas

Reconhecem-se, desde o desenho experimental, limitações que condicionam o escopo dos resultados:

- **Um único modelo e uma única base de dados.** Os experimentos empregaram exclusivamente o GPT-2 *small* sobre o WikiText-2. Os resultados podem não se transferir diretamente para modelos de escala substancialmente maior ou para bases de dados de natureza distinta. A implementação foi mantida parametrizável justamente para viabilizar essa verificação em trabalhos futuros.
- **Duas estratégias de poda, e não mais.** A restrição de tempo experimental levou à escolha de duas estratégias que cobrem os extremos de granularidade. Métodos baseados em informação de gradiente ou de segunda ordem ficaram fora do escopo.
- **Faixa restrita de taxas-sonda no perfil de sensibilidade.** O perfil que alimenta a alocação foi levantado a uma única taxa, e sua estabilidade foi verificada por meio de uma réplica a uma segunda taxa (Seção 6.4.5). Ambas pertencem, porém, ao mesmo regime intermediário de poda: nada garante que a ordenação de sensibilidade entre camadas se mantenha em regimes muito mais brandos ou muito mais agressivos.
- **Sensibilidade medida em isolamento.** A sonda mede o efeito de cada camada isoladamente e ignora interações entre elas. Como se verifica no Capítulo 6, a degradação é fortemente superaditiva, de modo que o perfil deve ser lido como ordenação relativa de importância, e não como modelo preditivo do desempenho de uma configuração completa.
- **Dependência do *hardware*.** Tempo e energia são intrinsecamente dependentes da máquina. Os valores absolutos não devem ser comparados entre ambientes distintos; a análise apoia-se em comparações relativas dentro de um mesmo ambiente.
- **Hiperparâmetros não otimizados.** Os hiperparâmetros do *fine-tuning* foram tomados de valores recomendados na literatura (Seção 5.5.4), sem busca sistemática. Valores melhores podem existir.
- **Semente única no *fine-tuning*.** Esta é a única etapa estocástica do experimento, e todas as configurações foram executadas com uma única semente. As perplexities

pós-retreinamento não vêm, portanto, acompanhadas de estimativa de variância, e diferenças pequenas entre configurações retreinadas devem ser lidas com a cautela correspondente. A comparação principal da etapa — alocação guiada contra alocação global — é afetada por essa limitação justamente no orçamento em que as duas praticamente empatam.

- **Estimativa de emissões.** A conversão de energia em emissões depende de tabelas regionais de intensidade de carbono, que carregam incerteza e variaram entre as sessões de execução. As emissões devem ser lidas como aproximações de ordem de grandeza.

Apesar dessas limitações, o desenho descrito é suficiente para responder às duas perguntas enunciadas na Seção 5.1, que são o objeto do Capítulo 6.

6 Resultados

Este capítulo apresenta os resultados experimentais obtidos com o GPT-2 *small* sobre o WikiText-2. A exposição segue a ordem em que os experimentos foram conduzidos: primeiro as duas estratégias clássicas de poda — magnitude (Seção 6.2) e estruturada (Seção 6.3) —, cujos resultados delimitam o problema; em seguida a estratégia otimizada proposta, em suas duas fases: o levantamento do perfil de sensibilidade por camada (Seção 6.4) e a varredura da alocação dele derivada (Seção 6.5). A Seção 6.6 submete as configurações mais relevantes ao *fine-tuning* de recuperação, e a Seção 6.7 sintetiza os achados.

6.1 Protocolo e Modelo de Referência

Todos os experimentos partem do GPT-2 *small* pré-treinado (124.439.808 parâmetros), avaliado no conjunto de teste do WikiText-2 por janela deslizante com *stride* de 512 *tokens*. A perplexity do modelo denso é de **25,17**. O valor é inferior aos 29,41 publicados por Radford et al. (2019), e a diferença é compatível com a do protocolo de avaliação: com janelas deslizantes sobrepostas, cada *token* é predito com pelo menos 512 *tokens* de contexto, o que reduz a *perplexity* em relação à avaliação em janelas disjuntas. A documentação da biblioteca `transformers` registra esse efeito no GPT-2 *large*: 19,44 com janelas disjuntas de 1.024 *tokens*, valor próximo dos 19,93 publicados, e 16,44 com *stride* de 512 (Hugging Face, 2026), redução de cerca de 15%. Redução da mesma ordem, cerca de 14%, separa os 29,41 publicados para o *small* dos 25,17 medidos aqui, com o mesmo protocolo daquela documentação. As medições de custo do modelo denso são: 474,70 MB de memória de parâmetros e 126,6 GFLOPs por janela de 512 *tokens*.

Duas ressalvas metodológicas condicionam a leitura das tabelas que se seguem.

A primeira é que **todos os resultados das varreduras são *one-shot***, com exceção da Seção 6.6, dedicada justamente ao retreinamento: poda-se o modelo pré-treinado e mede-se imediatamente, sem qualquer *fine-tuning* de recuperação. Essa escolha isola o efeito da poda do efeito do retreinamento, mas implica que as perplexities aqui reportadas são substancialmente piores que as da literatura, em que a poda é seguida de retreinamento. As curvas não devem, portanto, ser lidas como proposta de configuração de uso, e sim como medida do dano que cada estratégia de alocação causa por si só. A Seção 6.6 mostra que esse dano é, em boa parte, reversível.

A segunda é que as varreduras foram executadas em sessões distintas do ambiente Kaggle, e as medições de **custo dependem da sessão**. O mesmo modelo denso registrou

40,5 ms por janela em uma sessão, 54,0 ms em outra e 46,3 ms em uma terceira; o consumo de energia do modelo denso variou de forma análoga, entre 1176 e 1572 μWh. Por essa razão, comparações de tempo e energia **entre** varreduras são sempre apresentadas em termos relativos ao *baseline* da própria sessão, nunca em valores absolutos. As emissões de CO_2, que dependem adicionalmente do fator de carbono da rede elétrica do *datacenter* — que também variou entre sessões —, não são comparadas entre varreduras.

6.2 Poda por Magnitude

A Tabela 9 apresenta a varredura da poda por magnitude (HAN et al., 2015) nos dois escopos de alocação implementados: *global*, em que um único limiar sobre o valor absoluto dos pesos é aplicado a toda a rede, e *uniforme*, em que cada camada recebe a mesma fração de poda, com limiar próprio.

Tabela 9 – Poda por magnitude *one-shot* no GPT-2 *small* / WikiText-2. As colunas de custo referem-se ao escopo uniforme; o escopo global produz valores idênticos, pois nenhum dos dois altera a arquitetura.

Esparsidade	**Perplexity**		**Custo**		
	Global	**Uniforme**	**Params.** $\neq 0$	**Tamanho**	**FLOPs**
0 %	25,17	25,17	124,4 M	474,7 MB	126,6 G
10 %	25,40	25,47	115,9 M	474,7 MB	126,6 G
20 %	26,87	27,22	107,5 M	474,7 MB	126,6 G
30 %	40,23	36,42	99,0 M	474,7 MB	126,6 G
40 %	165,20	104,82	90,5 M	474,7 MB	126,6 G
50 %	1096,50	473,28	82,0 M	474,7 MB	126,6 G
60 %	5121,25	2004,20	73,5 M	474,7 MB	126,6 G
70 %	9371,16	6891,85	65,0 M	474,7 MB	126,6 G
80 %	8847,27	10871,00	56,5 M	474,7 MB	126,6 G
90 %	4612,69	13996,88	48,0 M	474,7 MB	126,6 G

O resultado mais importante da tabela está nas três últimas colunas, e é negativo: **o custo não cai**. Entre 0 % e 90 % de esparsidade, o número de parâmetros não nulos cai de 124,4 M para 48,0 M — uma redução de 61 % —, enquanto o tamanho em memória e os FLOPs permanecem rigorosamente constantes: 474,70 MB e 126,58 G em todas as dez configurações, valores idênticos dígito a dígito. A explicação é direta: a poda não estruturada substitui pesos por zeros, mas não remove nada da arquitetura. Os tensores mantêm as mesmas dimensões, e a GPU multiplica os zeros exatamente como multiplicava os valores originais. O ganho de 61 % em parâmetros é contábil, não físico.

As grandezas medidas empiricamente — tempo e energia — comportam-se de modo ligeiramente diferente das calculadas: exibem uma deriva descendente pequena e

monótona ao longo da varredura, de 54,0 para 51,0 ms e de 1572 para 1432 μWh, isto é, 5,6 % e 8,9 % entre os extremos. Essa deriva é da ordem da variação observada entre execuções repetidas do mesmo modelo denso — que registrou de 40,5 a 54,0 ms e de 1176 a 1572 μWh conforme a sessão, como discutido na Seção 6.1 — e não pode ser atribuída à poda, uma vez que a arquitetura executada é literalmente a mesma em todos os níveis. Serve, na verdade, como calibração da magnitude do ruído de medição: qualquer efeito de poda inferior a cerca de 10 % em tempo ou energia está abaixo do limiar de detecção deste protocolo. Os 55,5 % de redução de tempo que a poda estruturada registra na próxima seção estão uma ordem de grandeza acima desse limiar.

Esse achado responde empiricamente, e pela negativa, à primeira das duas perguntas que orientam este trabalho — a de se a redução teórica de parâmetros se converte em redução real de consumo. Na Tesla T4 utilizada, e sem suporte de *hardware* ou de *kernels* para esparsidade, a poda não estruturada não entrega ganho de eficiência algum.

Quanto à qualidade, a curva de perplexity apresenta um joelho acentuado em torno de 30 %. Até 20 % de esparsidade a poda é praticamente gratuita (acréscimo de 0,3 a 2,1 pontos de perplexity); a partir de 40 % a degradação torna-se explosiva. A região utilizável em regime *one-shot* é, portanto, limitada a esparsidades de até 20 %.

Por fim, a comparação entre os dois escopos revela um cruzamento duplo: os escopos empatam até 20 %, o uniforme domina claramente entre 30 % e 70 % — exatamente a faixa em que a escolha de alocação teria consequência prática — e o global só apresenta valores menores em 80 % e 90 %, região em que ambos os modelos já estão destruídos (perplexity acima de 4600) e a comparação perde sentido. Na faixa que importa, portanto, a alocação uniforme é preferível à global. Esse é um resultado contraintuitivo: o escopo global é o que aloca o orçamento com base em uma medida de importância — a magnitude do peso — e ainda assim perde para a alocação que ignora qualquer medida de importância. Trata-se do primeiro indício de que a magnitude do peso é um *proxy* pobre para a importância de um parâmetro.

6.3 Poda Estruturada

A Tabela 10 apresenta a varredura da poda estruturada (LI et al., 2017), adaptada ao *Transformer*: remoção física de cabeças de atenção e de neurônios das camadas MLP, ordenados por norma L_1. A esparsidade-alvo é expressa como fração de *estruturas* removidas; a fração de parâmetros efetivamente eliminada aparece como esparsidade real.

O contraste com a Tabela 9 é o achado central deste trabalho no plano da eficiência. Aqui **o custo cai, e cai proporcionalmente aos parâmetros**: a 90 % de estruturas removidas, o tempo de inferência cai 56 %, a energia 59 %, o tamanho em memória 61 % e os FLOPs 62 %, acompanhando a redução de 61 % no número de parâme-

Tabela 10 – Poda estruturada *one-shot* (cabeças e neurônios MLP, escopo global) no GPT-2 *small* / WikiText-2. O tempo de inferência é normalizado pelo *baseline* da mesma sessão (40,5 ms).

Estrut.	**Perplexity**	**Params.**	**Tamanho**	**FLOPs**	**Tempo rel.**	**Energia rel.**
0 %	25,17	124,4 M	474,7 MB	126,6 G	1,000	1,000
10 %	356,20	116,0 M	442,6 MB	118,0 G	0,968	0,975
20 %	2102,06	107,6 M	410,5 MB	109,3 G	0,908	0,945
30 %	2413,52	99,0 M	377,6 MB	100,5 G	0,831	0,865
40 %	2434,92	90,6 M	345,5 MB	91,9 G	0,756	0,786
50 %	2402,58	81,9 M	312,6 MB	83,1 G	0,688	0,714
60 %	2957,26	73,5 M	280,5 MB	74,5 G	0,628	0,638
70 %	4635,36	65,1 M	248,3 MB	65,9 G	0,557	0,558
80 %	7620,02	56,5 M	215,5 MB	57,0 G	0,503	0,489
90 %	5822,89	48,1 M	183,3 MB	48,4 G	0,445	0,413

tros. As duas tabelas formam, juntas, a resposta completa à primeira pergunta empírica: a redução de parâmetros só se converte em redução de consumo quando a poda remove estruturas inteiras da arquitetura, e não quando apenas as zera.

O preço dessa eficiência, contudo, é severo em regime *one-shot*. Já com 10 % de estruturas removidas a perplexity salta de 25,17 para 356,20 — compare-se com 25,40 na poda por magnitude ao mesmo nível. **Não existe região gratuita** na poda estruturada sem retreinamento, o que é coerente com o procedimento original de Li et al. (2017), em que a poda é sempre seguida de *fine-tuning*.

Dois resultados secundários merecem registro. O primeiro é que a inversão de escopo observada na magnitude se repete ao contrário: na poda estruturada é o escopo **global** que domina, e a alocação uniforme colapsa em taxas altas (perplexity de 53.313 a 80 % e de 827.316 a 90 %, contra 7620 e 5823 do global). Somado ao achado da seção anterior, isso estabelece que **nenhuma das duas alocações ingênuas vence nos dois regimes** — a constatação que motiva diretamente a estratégia otimizada.

O segundo é a assimetria entre os dois tipos de estrutura. Remover cabeças de atenção é notavelmente mais tolerável que remover neurônios MLP: com 129 das 144 cabeças removidas (90 %), a perplexity fica em 781, ao passo que a remoção equivalente de neurônios MLP produz valores acima de 3900. A curva das cabeças é, ainda, não monotônica, com mínimo local em torno de 30 % (perplexity 162, inferior aos 237 obtidos com apenas 10 % removidas) — eco direto do achado de Michel, Levy e Neubig (2019) sobre a redundância das cabeças de atenção. Em contrapartida, as camadas MLP concentram cerca de dois terços dos parâmetros podáveis, de modo que a poda restrita a cabeças tem alcance limitado como estratégia de compressão.

6.4 Perfil de Sensibilidade por Camada

As duas seções anteriores mostraram que nenhuma alocação ingênua do orçamento de poda domina o espectro de esparsidade: na poda por magnitude o escopo uniforme prevaleceu na faixa útil (Seção 6.2), enquanto na poda estruturada foi o escopo global que obteve as melhores perplexidades (Seção 6.3) — e ambas as alocações colapsam em algum regime. A hipótese central deste trabalho é que a causa é comum aos dois casos: as camadas do *Transformer* não são igualmente importantes, de modo que tanto tratá-las de forma idêntica (alocação uniforme) quanto confiar exclusivamente na magnitude dos pesos como proxy de importância (alocação global) distribuem mal o orçamento disponível.

Esta seção reporta o experimento que testa diretamente essa hipótese. Para cada uma das 12 camadas do GPT-2 *small* e para cada uma das duas estratégias, poda-se **apenas** aquela camada a uma taxa-sonda fixa de 50 %, mantendo-se as demais intactas, e mede-se a perplexity resultante no conjunto de teste do WikiText-2. Não há *fine-tuning* — o procedimento é *one-shot*, como nas varreduras anteriores. A degradação é registrada em escala logarítmica,

$$\Delta_\ell = \ln\left(\frac{\mathrm{PPL}_\ell}{\mathrm{PPL}_{\mathrm{base}}}\right), \tag{6.1}$$

onde PPL_ℓ é a perplexity do modelo com apenas a camada ℓ podada e $\mathrm{PPL}_{\mathrm{base}} = 25{,}17$ é a perplexity do modelo denso. O logaritmo é necessário porque a perplexity de modelos podados sem recuperação varia por ordens de grandeza: em escala linear, uma única camada catastrófica tornaria as demais indistinguíveis entre si. O experimento totaliza 24 medições (12 camadas $\times$ 2 estratégias).

6.4.1 Perfil Obtido

A Tabela 11 apresenta os resultados e a Figura 8 os representa graficamente.

O primeiro resultado é que a hipótese se confirma de forma inequívoca: a sensibilidade está longe de ser uniforme. Sob poda por magnitude, a camada mais frágil degrada o modelo 11,3 vezes mais que a mais robusta; sob poda estruturada, 42,9 vezes mais. Tratar as 12 camadas como intercambiáveis — que é exatamente o que a alocação uniforme faz — ignora uma diferença de uma ordem de grandeza no custo de podar cada uma delas.

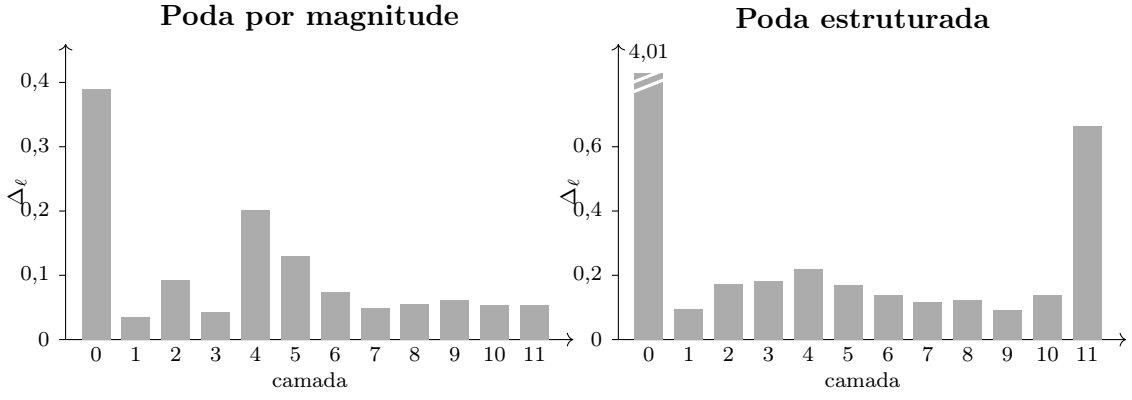
6.4.2 A Camada 0 como Gargalo

O segundo resultado é a magnitude do papel da primeira camada. Nas duas estratégias ela é a mais sensível, mas o efeito é qualitativamente distinto no caso estrutu-

Tabela 11 – Perfil de sensibilidade por camada do GPT-2 *small* no WikiText-2. Cada linha corresponde a podar exclusivamente a camada indicada, à taxa-sonda de 50 %, sem *fine-tuning*. Perplexity do modelo denso: 25,17.

	Poda por magnitude		**Poda estruturada**	
Camada	**Perplexity**	$\boldsymbol{\Delta_\ell}$	**Perplexity**	$\boldsymbol{\Delta_\ell}$
0	37,17	0,390	1386,43	4,009
1	26,06	0,035	27,66	0,094
2	27,62	0,093	29,91	0,173
3	26,28	0,043	30,23	0,183
4	30,81	0,202	31,31	0,218
5	28,66	0,130	29,80	0,169
6	27,09	0,074	28,90	0,138
7	26,44	0,049	28,31	0,118
8	26,59	0,055	28,45	0,122
9	26,76	0,061	27,63	0,093
10	26,55	0,054	28,89	0,138
11	26,55	0,054	48,87	0,663
Razão máx./mín.	11,3×		42,9×	

Figura 8 – Degradação Δ_ℓ causada pela poda isolada de cada camada, à taxa-sonda de 50 %. Note-se que as escalas verticais diferem entre os dois gráficos; na poda estruturada, a barra da camada 0 excede o eixo e está truncada.



Fonte: Elaborada pelo autor.

rado: remover metade das cabeças de atenção e dos neurônios MLP **apenas do bloco 0** eleva a perplexity de 25,17 para 1386,43, ou seja, 55 vezes o valor do modelo denso. Para dimensionar esse número, a varredura estruturada uniforme a 50 % — que poda *todas* as 12 camadas — atinge perplexity 4573,00. Em escala logarítmica, a camada 0 sozinha responde por $4{,}01/5{,}20 \approx 77\,\%$ de toda a degradação produzida pela poda uniforme do modelo inteiro.

A interpretação é coerente com o papel funcional do primeiro bloco. É nele que

as representações deixam de ser a soma direta de *embeddings* de token e de posição e passam a ser contextualizadas pela primeira vez; qualquer informação descartada ali não tem como ser recuperada pelas camadas subsequentes, que operam sobre um sinal já degradado. Trata-se, portanto, de um erro que se propaga por toda a pilha, ao contrário de um erro cometido nas camadas intermediárias.

Na poda estruturada observa-se ainda um segundo ponto crítico na outra extremidade: a camada 11, imediatamente anterior à cabeça de linguagem, apresenta $\Delta_{11} = 0{,}663$, quase cinco vezes a mediana das camadas intermediárias. O perfil estruturado tem, assim, formato aproximado de "U": as bordas da pilha são críticas e o miolo é comparativamente barato. Esse padrão não se repete na poda por magnitude, cujo perfil decai após um pico inicial e mantém as últimas camadas entre as mais robustas.

6.4.3 Os Perfis Dependem da Estratégia

O terceiro resultado tem consequência metodológica direta: os dois perfis **não** são o mesmo. A correlação de postos de Spearman entre eles é $\rho = 0{,}50$ — há concordância, mas parcial. As duas estratégias coincidem em eleger a camada 0 como a mais crítica e em considerar as camadas 1 e 9 entre as mais robustas, e divergem em praticamente todo o resto: a camada 11 é a segunda mais sensível sob poda estruturada e apenas a oitava sob poda por magnitude; a camada 3 é a segunda mais robusta sob magnitude e a quarta mais sensível sob poda estruturada.

A implicação prática é que não existe um perfil de sensibilidade único do GPT-2. Sensibilidade é uma propriedade do par (camada, estratégia de poda), não da camada isoladamente, e o perfil precisa ser levantado separadamente para cada estratégia — como foi feito aqui — sob pena de se alocar o orçamento com um mapa que pertence a outro método. Esse achado também oferece uma explicação para a inversão observada nas varreduras ingênuas: como magnitude de peso e norma de estrutura respondem a aspectos diferentes da importância de uma camada, é esperado que o escopo global (que decide com base nessas grandezas) acerte em um caso e erre no outro.

6.4.4 Da Sensibilidade à Alocação

O perfil é um diagnóstico; para virar estratégia, precisa ser convertido em uma taxa de poda por camada. Essa conversão é a Equação 5.2, apresentada na Seção 5.4.2: a taxa de cada camada é proporcional à sua robustez — o inverso da degradação medida — elevada ao expoente β, escalada por um fator único obtido por bisseção de modo que o orçamento global seja respeitado, e limitada pelo teto $r_{\max} = 0{,}95$. Interessa aqui apenas o que ela produz na prática.

A Tabela 12 mostra as taxas que a Equação 5.2 produziu para um orçamento

global de 50 % com $\beta = 1$. O contraste com a alocação uniforme é acentuado: sob poda estruturada, a camada 0 recebe 2 % de poda e a camada 1 recebe 84 %, enquanto a alocação uniforme aplicaria 50 % a ambas.

Tabela 12 – Taxas de poda por camada efetivamente aplicadas pela Equação 5.2 para orçamento global de 50 % e $\beta = 1$ (coluna `taxas_por_camada` dos registros experimentais). A alocação uniforme corresponderia a 50 % em todas as camadas.

Camada	0	1	2	3	4	5	6	7	8	9	10	11
Magnitude (%)	8	94	35	75	16	25	44	67	59	53	61	61
Estruturada (%)	2	84	46	43	36	47	57	67	64	85	57	12

6.4.5 Robustez do Perfil à Taxa-Sonda

O perfil apresentado até aqui foi levantado a uma taxa-sonda de 50 %. Como a escolha dessa taxa é arbitrária, cabe verificar se a ordenação de sensibilidade que dela resulta é uma propriedade das camadas ou um artefato do regime de poda em que a sonda opera. Para isso, o perfilamento completo foi repetido a uma taxa-sonda de 30 %, nas duas estratégias, o que fornece uma réplica independente da fase 1. A Tabela 13 confronta os dois levantamentos.

Tabela 13 – Perfis de sensibilidade levantados a duas taxas-sonda. Os valores são os Δ_ℓ da Equação 5.1; entre parênteses, o posto da camada na ordenação de sensibilidade (1 = mais sensível).

	Magnitude		**Estruturada**	
Camada	**Sonda 30 %**	**Sonda 50 %**	**Sonda 30 %**	**Sonda 50 %**
0	0,0414 (1)	0,3899 (1)	3,4235 (1)	4,0088 (1)
1	0,0067 (12)	0,0346 (12)	0,1593 (3)	0,0944 (11)
2	0,0121 (7)	0,0929 (4)	0,1302 (7)	0,1726 (5)
3	0,0074 (11)	0,0433 (11)	0,1426 (5)	0,1832 (4)
4	0,0390 (2)	0,2023 (2)	0,1560 (4)	0,2184 (3)
5	0,0346 (3)	0,1298 (3)	0,0929 (8)	0,1688 (6)
6	0,0164 (4)	0,0735 (5)	0,1320 (6)	0,1383 (7)
7	0,0077 (10)	0,0490 (10)	0,0526 (12)	0,1176 (10)
8	0,0107 (8)	0,0549 (7)	0,0650 (10)	0,1225 (9)
9	0,0137 (5)	0,0611 (6)	0,0596 (11)	0,0934 (12)
10	0,0078 (9)	0,0535 (9)	0,0885 (9)	0,1378 (8)
11	0,0132 (6)	0,0535 (8)	0,5387 (2)	0,6635 (2)

Fonte: Elaboração própria.

A correlação de postos de Spearman entre as duas sondas é $\rho = 0{,}944$ na poda por magnitude e $\rho = 0{,}713$ na poda estruturada. A ordenação é, portanto, substancialmente estável — e os achados que sustentam a estratégia otimizada sobrevivem à troca de sonda. Na magnitude, as três camadas mais sensíveis (0, 4 e 5) e as três mais robustas (1, 3 e 7) são **as mesmas** nos dois levantamentos. Na estruturada, a camada 0 e a camada 11 ocupam os dois primeiros postos em ambos, de modo que o formato em "U" discutido na Seção 6.4.2 não depende da taxa-sonda; a camada 0 responde por 65,5 % da soma dos Δ_ℓ a 50 % e por 67,9 % a 30 %.

A estabilidade é menor na poda estruturada, e a divergência concentra-se em uma camada: a camada 1 salta do 11º posto na sonda de 50 % para o 3º na de 30 %. A leitura mais plausível é que o miolo da pilha tem sensibilidades próximas entre si — os Δ_ℓ das camadas 1 a 10 ficam todos entre 0,05 e 0,22 —, de modo que pequenas variações de medição reordenam postos sem alterar a estrutura do perfil. Isso é coerente com o comportamento do expoente β reportado na Seção 6.5.4: o que a alocação efetivamente aproveita é o sinal forte das bordas, não a ordenação fina do miolo.

Registre-se que o perfil de 30 % foi mantido em arquivo separado e não alimentou nenhuma das alocações reportadas: todas as varreduras da Seção 6.5 usam exclusivamente o perfil de 50 %. A réplica cumpre aqui o papel de verificação, não de insumo.

6.4.6 Limitações do Perfilamento

Três limitações do procedimento devem ser explicitadas, pois condicionam a leitura dos resultados da varredura otimizada.

A primeira é que, embora a verificação da seção anterior mostre que a ordenação é estável entre 30 % e 50 %, ambas as taxas pertencem ao mesmo regime intermediário de poda. Nada garante que a ordenação se mantenha em regimes extremos — poda muito branda, em que a degradação de todas as camadas se aproxima do ruído de medição, ou muito agressiva, em que várias camadas já colapsaram. Estender a verificação a essas faixas tem custo proporcional: cada ponto de perfil exige uma avaliação completa de perplexity.

A segunda, e mais relevante, é que a sonda mede o efeito de cada camada **isoladamente**, ignorando as interações entre elas. Os dados do próprio experimento mostram que essa aproximação é imperfeita: a soma dos Δ_ℓ individuais sob poda por magnitude é 1,238, ao passo que a poda uniforme simultânea de todas as camadas a 50 % produz degradação logarítmica de 2,934 — mais que o dobro. A degradação é fortemente superaditiva, isto é, os erros introduzidos em camadas distintas se compõem em vez de simplesmente se somarem. O perfil deve, por isso, ser lido como uma ordenação relativa de importância, e não como um modelo preditivo da degradação de uma configuração completa.

A terceira é que todas as medições são *one-shot*, sem *fine-tuning* de recuperação. Camadas que parecem frágeis nesse regime poderiam ser recuperáveis com retreinamento, o que eventualmente alteraria a alocação ótima. A opção pelo regime *one-shot* é deliberada e coerente com o objetivo do trabalho — isolar o efeito da poda do efeito do retreinamento —, mas restringe a generalidade das conclusões.

Nenhuma dessas limitações invalida o achado principal desta seção, que é robusto às três: sob qualquer leitura, as camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, e essa diferença é informação que as alocações uniforme e global simplesmente não utilizam. Se essa informação se converte em ganho efetivo na fronteira entre compressão e qualidade é o que a varredura da estratégia otimizada, reportada na seção seguinte, se propõe a verificar.

6.5 Varredura da Alocação Guiada por Sensibilidade

Esta seção reporta a fase que responde à pergunta central do trabalho: a alocação derivada do perfil de sensibilidade melhora, de fato, o compromisso entre compressão e qualidade em relação às alocações ingênuas? As taxas por camada produzidas pela Equação 5.2 foram aplicadas nos mesmos dez níveis de orçamento das varreduras anteriores, para as duas estratégias, e avaliadas com o mesmo vetor de métricas. O experimento foi replicado com três valores do expoente β — 1,0, 0,5 e 0,0 —, o que fornece a ablação da função de alocação sobre um mesmo perfil. O caso $\beta = 0$ não é uma alocação candidata, e sim um controle: nele a regra atribui a mesma taxa a todas as camadas e deve, por construção, reproduzir a poda uniforme (Seção 6.5.3).

6.5.1 Poda por Magnitude

A Tabela 14 compara as quatro alocações.

A alocação guiada vence em oito dos nove níveis, e as margens são substanciais na faixa de interesse. Em 30 % de esparsidade, a perplexity cai de 36,42 (melhor alocação ingênua) para 31,20 — uma redução de 14 %. Em 40 %, o ganho é de 104,82 para 57,30, ou seja, a degradação é praticamente **reduzida à metade** usando exatamente o mesmo orçamento de poda. Em 50 % a vantagem persiste, ainda que modesta (447,57 contra 473,28). Nos níveis altos, de 60 % a 80 %, a configuração com $\beta = 1$ produz perplexities entre 2,5 e 4 vezes menores que a melhor alocação ingênua — embora, nesse regime, todos os modelos já estejam degradados a ponto de a comparação ter interesse mais teórico que prático.

A única derrota ocorre em 90 %, onde o escopo global apresenta o menor valor. Como discutido na Seção 6.2, essa é a região em que a perplexity de todas as configu-

Tabela 14 – Perplexity da poda por magnitude sob as quatro alocações. Em negrito, o menor valor de cada linha.

	Alocação ingênua		**Guiada por sensibilidade**	
Esparsidade	**Global**	**Uniforme**	$\boldsymbol{\beta = 1{,}0}$	$\boldsymbol{\beta = 0{,}5}$
10 %	**25,40**	25,47	25,44	25,44
20 %	26,87	27,22	26,79	**26,57**
30 %	40,23	36,42	34,62	**31,20**
40 %	165,20	104,82	193,83	**57,30**
50 %	1096,50	473,28	1026,09	**447,57**
60 %	5121,25	2004,20	**1524,55**	2171,79
70 %	9371,16	6891,85	**1853,30**	2805,39
80 %	8847,27	10871,00	**2747,56**	3715,68
90 %	**4612,69**	13996,88	10398,10	7616,63

rações ultrapassa 4600 e a ordenação entre modelos igualmente inservíveis não carrega significado.

6.5.2 Poda Estruturada

Na poda estruturada, os ganhos são de outra ordem de grandeza, como mostra a Tabela 15.

Tabela 15 – Perplexity da poda estruturada sob as quatro alocações. Em negrito, o menor valor de cada linha.

	Alocação ingênua		**Guiada por sensibilidade**	
Estruturas	**Global**	**Uniforme**	$\boldsymbol{\beta = 1{,}0}$	$\boldsymbol{\beta = 0{,}5}$
10 %	356,20	481,13	**124,29**	215,83
20 %	2102,06	1484,89	**371,07**	717,19
30 %	2413,52	5467,56	**724,53**	923,65
40 %	2434,92	10486,71	**1301,15**	9489,87
50 %	2402,58	4573,00	**2327,59**	7922,81
60 %	**2957,26**	5799,76	5393,15	12322,05
70 %	**4635,36**	12249,58	10130,33	103463,60
80 %	**7620,02**	53313,38	8891,77	203995,84
90 %	5822,89	827316,19	5353,85	**4830,24**

Na faixa de 10 % a 50 % de estruturas removidas, a alocação guiada com $\beta = 1$ domina de forma inequívoca. Em 10 %, a perplexity cai de 356,20 para 124,29 — uma redução de 65 %. Em 20 %, de 1484,89 para 371,07, um fator de **quatro**. Em 30 %, de 2413,52 para 724,53, um fator de **3,3**. Considerando que a Seção 6.3 concluiu que não

existe região gratuita para a poda estruturada *one-shot*, reduzir a perplexity a um quarto no mesmo orçamento é o resultado mais expressivo deste capítulo.

Uma ressalva quantitativa é necessária. Como a remoção de estruturas é discreta, as taxas fracionárias por camada não são atingíveis exatamente, e a alocação guiada acaba removendo uma fração de parâmetros ligeiramente menor que a ingênua no mesmo alvo: em 10 %, esparsidade real de 9,20 % contra 9,90 % do escopo global; em 30 %, 29,01 % contra 29,95 %. Parte da vantagem observada deve-se, portanto, a um corte marginalmente menor. A diferença é de menos de um ponto percentual de esparsidade, ao passo que a diferença de perplexity chega a um fator de quatro, de modo que a explicação não se sustenta como causa principal — mas o efeito existe e deve ser declarado.

A partir de 60 %, a ordenação se inverte e o escopo global volta a vencer. A causa é identificável nas taxas alocadas: a partir desse orçamento, um número crescente de camadas satura no teto $r_{\max} = 0{,}95$ — duas camadas em 60 %, quatro em 70 %, oito em 80 % e onze em 90 %. Saturadas as camadas, a alocação deixa de discriminar entre elas e degenera em algo próximo da poda uniforme sobre o subconjunto saturado, ao mesmo tempo em que concentra uma pressão crescente sobre as poucas camadas ainda livres. A estratégia guiada, portanto, **só é informativa enquanto o orçamento couber dentro do teto por camada**; acima disso, ela perde sua vantagem estrutural. Esse é um limite intrínseco de qualquer alocação proporcional com teto, e não um defeito de implementação.

6.5.3 O Controle $\beta = 0$

Antes de interpretar a ablação, cabe verificar se a regra de alocação está correta. Com $\beta = 0$ o fator $\Delta_\ell^{-\beta}$ da Equação 5.2 vale 1 para toda camada, de modo que todas recebem a mesma taxa e a alocação deve coincidir com a varredura uniforme correspondente. É um teste de sanidade barato, pois reaproveita o perfil já levantado.

Na poda estruturada a coincidência é **exata**: nos dez níveis de orçamento, a alocação com $\beta = 0$ reproduz a perplexity da varredura uniforme até a última casa decimal registrada — 5467,5635 em 30 %, 4573,0020 em 50 % e 827.316,1875 em 90 %, para citar três níveis. A identidade confirma que a conversão de perfil em taxas, a bisseção do fator de escala e a aplicação das taxas por camada estão implementadas corretamente: o que a alocação guiada obtém a mais, nas seções anteriores, vem do perfil e não de alguma diferença espúria de procedimento.

Na poda por magnitude a coincidência **não** ocorre, e a razão é uma diferença deliberada de implementação que merece registro. A varredura uniforme decide um limiar de magnitude para cada matriz de peso isoladamente, ao passo que a alocação por camada decide um limiar único por bloco, compartilhado pelas quatro projeções que o compõem.

As duas produzem a mesma esparsidade agregada — exatamente 30 %, 50 % e assim por diante —, mas distribuída de forma diferente entre as projeções de um mesmo bloco, e as perplexities divergem: 473,28 contra 590,25 em 50 %, e 13.996,88 contra 8610,96 em 90 %. A alocação por sensibilidade com $\beta = 0$ não é, portanto, a poda uniforme da Seção 6.2, e sim uma terceira alocação — "global dentro da camada" — cuja comparação com as demais fica fora do escopo deste trabalho. Para a poda estruturada, que é a estratégia efetivamente capaz de reduzir consumo, o controle vale integralmente.

6.5.4 Ablação do Expoente β

A comparação entre $\beta = 1{,}0$ e $\beta = 0{,}5$ produz um resultado que não era antecipado: **o valor adequado de β depende da estratégia**, e depende dela de forma oposta nas duas.

Na poda estruturada, $\beta = 1$ domina em todos os níveis até 80 %, e o amortecimento é francamente prejudicial — em 70 % e 80 % a configuração com $\beta = 0{,}5$ produz perplexities de 103.463 e 203.996, uma ordem de grandeza pior que qualquer outra alocação testada. Na poda por magnitude ocorre o inverso na faixa útil: $\beta = 0{,}5$ é melhor de 20 % a 50 %, e só a partir de 60 % o expoente pleno passa à frente.

A explicação mais plausível liga-se à dispersão dos perfis. O perfil estruturado tem razão máximo/mínimo de 42,9× e é dominado pela camada 0, cuja fragilidade é tão extrema (Seção 6.4.2) que qualquer amortecimento da alocação lhe devolve orçamento de poda com consequências catastróficas: em 70 %, $\beta = 0{,}5$ atribui 14,6 % de poda à camada 0, contra 3,0 % de $\beta = 1$. Quando o perfil contém um sinal dessa intensidade, convém segui-lo integralmente. Já o perfil de magnitude tem dispersão de 11,3×, distribuída entre várias camadas de sensibilidade intermediária; nele, a razão entre camadas é mais suscetível a ruído de medição, e amortecê-la funciona como regularização. O expoente β comporta-se, assim, como um parâmetro de confiança no perfil: quanto mais nítido o sinal, maior o β que compensa adotar.

6.5.5 Efeito sobre o Custo

Do ponto de vista da eficiência, a alocação guiada preserva integralmente o comportamento da estratégia de base, o que era esperado, uma vez que ela altera a distribuição do orçamento, não sua natureza. Na poda por magnitude, tamanho, FLOPs e tempo permanecem constantes, pelo mesmo motivo discutido na Seção 6.2. Na poda estruturada, o custo cai proporcionalmente: normalizando pelo *baseline* da própria sessão, a configuração com $\beta = 1$ registra tempo relativo de 0,659 e energia relativa de 0,674 em 50 %, e 0,419 e 0,384 em 90 % — valores equivalentes, dentro da variação entre sessões, aos 0,688 e 0,714 (50 %) e 0,445 e 0,413 (90 %) do escopo global.

O achado prático é, portanto, que **o ganho de qualidade da alocação guiada é gratuito em termos de custo**: na faixa de 10 % a 50 % da poda estruturada, obtém-se uma perplexity de três a quatro vezes menor com essencialmente o mesmo tempo de inferência, o mesmo consumo de energia e o mesmo tamanho de modelo. É exatamente o deslocamento da fronteira entre compressão e qualidade que a segunda pergunta empírica do trabalho se propunha a investigar.

6.6 *Fine-tuning* de Recuperação

Todas as varreduras apresentadas até aqui operam em regime *one-shot*. Esta seção reporta a Etapa 4 (Seção 5.5), que incorpora o retreinamento ao experimento para medir quanto da degradação causada pela poda pode ser recuperada: as sete configurações definidas na Seção 5.5.1 foram submetidas ao *fine-tuning* de recuperação, com a perplexity medida antes e depois do retreinamento. A quantidade de retreinamento é idêntica para todas: uma época sobre o conjunto de treino do WikiText-2, 297 passos do otimizador, taxa de aprendizado de 5×10^{-5} e semente fixa. Diferentemente das demais, esta varredura foi executada integralmente em uma única sessão, de modo que suas medições de tempo e energia são diretamente comparáveis entre si.

Tabela 16 – *Fine-tuning* de recuperação (uma época, 297 passos). A última coluna expressa a perplexity pós-retreinamento como múltiplo da perplexity do modelo denso retreinado, e não do modelo denso original.

Configuração	**Esp. real**	**PPL *one-shot***	**PPL pós-FT**	**Razão**
Denso (controle)	—	25,17	18,68	1,00
Magnitude 30 % (uniforme)	30,00 %	36,42	20,54	1,10
Magnitude 50 % (uniforme)	50,00 %	473,28	30,27	1,62
Estruturada 10 % (global)	9,90 %	356,20	25,33	1,36
Sensibilidade 10 % ($\beta = 1$)	9,20 %	124,29	25,96	1,39
Estruturada 30 % (global)	29,95 %	2413,52	104,78	5,61
Sensibilidade 30 % ($\beta = 1$)	29,01 %	724,53	56,08	3,00

6.6.1 O Controle Denso Reenquadra a Leitura

O primeiro resultado da Tabela 16 não envolve poda alguma: o modelo denso, submetido à mesma quantidade de retreinamento, tem sua perplexity reduzida de 25,17 para **18,68**. São 26 % de ganho obtidos sem remover um único parâmetro, atribuíveis exclusivamente à adaptação de domínio — o GPT-2 foi pré-treinado no WebText, não no WikiText-2, e o retreinamento o aproxima da distribuição da base de dados de avaliação.

A consequência metodológica é considerável e justifica retroativamente a decisão de incluir o controle. Sem ele, a configuração de magnitude a 30 % pareceria ter *superado* o modelo original: 20,54 contra 25,17, uma leitura que sugeriria que a poda melhora o modelo. A comparação correta é contra 18,68, e diante desse valor a mesma configuração aparece pelo que de fato é — 10 % pior que o denso equivalente. Toda a coluna de razão da Tabela 16 adota essa referência, e é a única leitura que separa recuperação do dano da poda de mera adaptação à base de dados.

6.6.2 A Recuperação é Substancial, mas Desigual

O retreinamento recupera uma fração expressiva do dano em todas as configurações, e a magnitude dessa recuperação é inversamente proporcional à qualidade do ponto de partida. A poda estruturada a 30 %, que o regime *one-shot* deixara em perplexity 2413,52 — um modelo destruído para qualquer propósito prático —, retorna a 104,78. A poda por magnitude a 50 %, em 473,28, retorna a 30,27, isto é, a 1,62 vez o denso retreinado.

Esse comportamento qualifica de forma importante a leitura das seções anteriores. A perplexity *one-shot* mede o dano imediato da poda, mas superestima o dano permanente: configurações que pareciam inutilizáveis são, após uma quantidade modesta de retreinamento, recuperáveis a patamares utilizáveis. A região praticável da poda, delimitada em 20 % de esparsidade na Seção 6.2, estende-se consideravelmente quando o retreinamento é incluído.

Registre-se ainda que a hierarquia entre granularidades se mantém: a configuração de magnitude a 30 % atinge a menor perplexity pós-retreinamento de todas as configurações podadas (20,54), mas, pelas razões estabelecidas na Seção 6.2, não reduz tempo, energia nem memória. As configurações estruturadas a 30 %, essas sim, registram 40,82 ms e 1,285 mWh por janela contra 50,21 ms e 1,588 mWh do denso retreinado — uma redução de aproximadamente 19 % em ambos.

6.6.3 O *Fine-tuning* Corrói a Vantagem da Alocação Guiada

O achado mais desconfortável desta varredura diz respeito diretamente à contribuição central do trabalho. As duas últimas linhas de cada bloco da Tabela 16 confrontam a alocação guiada por sensibilidade com a alocação global, nos mesmos dois orçamentos, antes e depois do retreinamento.

Em 10 % de estruturas removidas, a vantagem **desaparece**. Em regime *one-shot*, a alocação guiada registrava perplexity 124,29 contra 356,20 da alocação global, uma vantagem de 2,87 vezes. Após o retreinamento, a relação se inverte: 25,96 contra 25,33, com ligeira vantagem para a alocação global. Em 30 %, a vantagem sobrevive, mas

reduzida quase à metade: o fator de 3,33 vezes do regime *one-shot* (724,53 contra 2413,52) torna-se 1,87 vez (56,08 contra 104,78).

A interpretação é que a alocação por sensibilidade otimiza uma quantidade específica — o dano imediato causado pela poda — e o retreinamento repara justamente o tipo de dano que ela evita. Quanto maior a quantidade de retreinamento, menor a importância de haver podado bem. Sob essa leitura, o perfil de sensibilidade é mais valioso quanto mais escassa for a quantidade de retreinamento disponível, e a vantagem tende a crescer com a severidade da poda, o que é consistente com sua persistência em 30 % e seu desaparecimento em 10 %.

O achado não invalida a contribuição, mas delimita seu escopo com precisão: em orçamentos de poda agressivos, a alocação guiada continua entregando perplexity aproximadamente duas vezes menor ao mesmo custo computacional — 40,28 ms e 1,307 mWh contra 40,82 ms e 1,285 mWh da alocação global em 30 %, valores equivalentes dentro da variação de medição. Em orçamentos leves, seguidos de retreinamento, a alocação ingênua é suficiente.

6.6.4 Verificações de Integridade e Custo

Duas verificações confirmam a correção do procedimento. A esparsidade das configurações de magnitude, medida ao final do treinamento, permanece em exatamente 30,0000 % e 50,0000 %, o que confirma que a máscara binária descrita na Seção 5.5 foi corretamente reaplicada após cada passo do otimizador e que o modelo não voltou a ser denso. As esparsidades reais das configurações guiadas por sensibilidade, por sua vez, ficam ligeiramente abaixo do alvo (9,20 % e 29,01 % contra 9,90 % e 29,95 % das configurações globais), diferença atribuível à granularidade discreta das estruturas removidas — uma cabeça de atenção ou um neurônio MLP são indivisíveis. A diferença favorece marginalmente a alocação guiada e está registrada aqui por transparência, ainda que inferior a um ponto percentual.

Quanto ao custo, o retreinamento consome 7,33 mWh no modelo denso, contra 1,59 mWh de uma avaliação completa do mesmo modelo — aproximadamente 4,6 vezes. Trata-se, porém, de um custo pago uma única vez, ao passo que a economia por inferência da poda estruturada se acumula a cada uso. A configuração estruturada a 30 % economiza 0,303 mWh por avaliação completa do conjunto de teste (1,285 contra 1,588 mWh), de modo que o custo do próprio retreinamento se amortiza em aproximadamente 24 avaliações equivalentes — um volume trivial para um modelo em uso continuado. Cabe explicitar a unidade: o consumo é medido em torno do bloco de avaliação inteiro, que percorre todo o conjunto de teste por janela deslizante, e não em torno de uma janela individual. Converter a economia para base por janela exigiria registrar o número de janelas de cada avaliação, o que não foi feito; a amortização fica, portanto, expressa na unidade

efetivamente medida.

A limitação principal desta varredura é a quantidade de retreinamento: uma única época. Não é possível afirmar, a partir destes dados, se a erosão da vantagem da alocação guiada se estabiliza no patamar observado em 30 % ou se prossegue com mais épocas até desaparecer também nesse orçamento. Trata-se da verificação mais imediata que este trabalho deixa em aberto.

6.7 Síntese dos Resultados

Os experimentos permitem responder às duas perguntas que orientaram o trabalho.

À primeira — se a redução teórica de parâmetros se converte em redução real de consumo — a resposta é condicional à granularidade da poda. A poda não estruturada por magnitude reduz em 61 % os parâmetros não nulos sem alterar o tamanho em memória nem os FLOPs, e sem produzir em tempo ou energia qualquer efeito distinguível do ruído de medição do próprio ambiente: na Tesla T4 utilizada, sem suporte de *hardware* para esparsidade, o ganho é puramente contábil. A poda estruturada, ao remover fisicamente cabeças de atenção e neurônios MLP, converte a mesma redução de 61 % em parâmetros em quedas de 56 % no tempo, 59 % na energia e 62 % nos FLOPs. Para o objetivo declarado deste trabalho — redução do consumo de energia —, apenas a poda estruturada é uma estratégia viável.

À segunda — se a alocação otimizada empurra a fronteira entre compressão e qualidade — a resposta é afirmativa, com escopo delimitado. As camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, com a camada 0 respondendo isoladamente por cerca de três quartos da degradação da poda uniforme; essa informação, que nenhuma alocação ingênua utiliza, converte-se em ganho mensurável quando incorporada à distribuição do orçamento. Na poda estruturada — a estratégia que efetivamente reduz consumo — a alocação guiada reduz a perplexity por fatores de três a quatro na faixa de 10 % a 50 % de estruturas removidas, sem custo adicional de tempo, energia ou memória. O escopo da conclusão é delimitado por três fronteiras: a saturação da alocação em orçamentos acima de 60 %; o fato de que o perfil de sensibilidade precisa ser levantado separadamente para cada estratégia de poda, por não ser transferível entre elas; e, sobretudo, a quantidade de retreinamento disponível.

Esta última fronteira merece registro explícito, por ser a que mais qualifica a contribuição. O *fine-tuning* de recuperação (Seção 6.6) repara justamente o tipo de dano que a alocação guiada evita, e por isso corrói sua vantagem: em 10 % de estruturas removidas a vantagem desaparece após uma época de retreinamento, e em 30 % ela se reduz de um fator de 3,33 para 1,87. A alocação por sensibilidade é, portanto, uma

contribuição cujo valor cresce com a severidade da compressão e decresce com a quantidade de retreinamento disponível — útil onde a poda é agressiva e o retreinamento, caro, e dispensável no regime oposto. Sobre a leitura de todos esses números pesa ainda o controle denso: o mero retreinamento do modelo sem poda reduz a perplexity de 25,17 para 18,68, e é contra esse valor, não contra o modelo pré-treinado original, que qualquer recuperação deve ser medida.

7 Considerações Finais

Referências

BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2015. Citado na página 11.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, v. 5, n. 2, p. 157–166, 1994. Citado 2 vezes nas páginas 6 e 11.

BERGSTRA, J.; BENGIO, Y. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, v. 13, p. 281–305, 2012. Citado na página 8.

BIDERMAN, S. et al. Pythia: A suite for analyzing large language models across training and scaling. In: *Proceedings of the 40th International Conference on Machine Learning*. [S.l.: s.n.], 2023. (PMLR, v. 202), p. 2397–2430. Citado 2 vezes nas páginas 36 e 37.

BLALOCK, D. et al. What is the state of neural network pruning? In: *Proceedings of Machine Learning and Systems (MLSys)*. [S.l.: s.n.], 2020. Citado 6 vezes nas páginas 17, 21, 23, 29, 33 e 44.

COURTY, B. et al. *CodeCarbon: estimate and track carbon emissions from your computing*. 2023. Disponível em: <https://github.com/mlco2/codecarbon>. Acesso em: 17 abr. 2026. Citado na página 53.

DENIL, M. et al. Predicting parameters in deep learning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2013. Citado 2 vezes nas páginas 16 e 20.

DENTON, E. L. et al. Exploiting linear structure within convolutional networks for efficient evaluation. In: *Advances in Neural Information Processing Systems (NIPS)*. [S.l.: s.n.], 2014. v. 27, p. 1269–1277. Citado na página 17.

DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. [S.l.]: Association for Computational Linguistics, 2019. p. 4171–4186. Citado 3 vezes nas páginas 36, 37 e 52.

FRANKLE, J.; CARBIN, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 22 e 23.

FRANTAR, E.; ALISTARH, D. SparseGPT: Massive language models can be accurately pruned in one-shot. In: *Proceedings of the 40th International Conference on Machine Learning*. [S.l.: s.n.], 2023. (PMLR, v. 202), p. 10323–10337. Citado 3 vezes nas páginas 36, 37 e 43.

GAO, L. et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. Citado 4 vezes nas páginas 36, 37, 38 e 39.

GOKASLAN, A.; COHEN, V. *OpenWebText Corpus*. 2019. Disponível em: <http://Skylion007.github.io/OpenWebTextCorpus>. Acesso em: 28 set. 2026. Citado 2 vezes nas páginas 38 e 39.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org/>. Citado 7 vezes nas páginas 2, 3, 4, 6, 8, 9 e 27.

HAN, S. et al. Learning both weights and connections for efficient neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2015. Citado 11 vezes nas páginas 17, 19, 23, 31, 33, 43, 44, 45, 49, 51 e 59.

HENDRYCKS, D.; GIMPEL, K. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. Citado na página 14.

HINTON, G.; VINYALS, O.; DEAN, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. Citado na página 17.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 11.

Hugging Face. *Perplexity of fixed-length models*. 2026. Documentação da biblioteca `transformers`. Disponível em: <https://huggingface.co/docs/transformers/perplexity>. Acesso em: 30 set. 2026. Citado 3 vezes nas páginas 25, 37 e 58.

JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2018. Citado na página 16.

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing*. 3. ed. [S.l.]: Stanford University, 2025. Rascunho da 3.ª edição. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Citado na página 9.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2012. Citado na página 5.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, n. 7553, p. 436–444, 2015. Citado 2 vezes nas páginas 2 e 6.

LECUN, Y.; DENKER, J. S.; SOLLA, S. A. Optimal brain damage. In: *Advances in Neural Information Processing Systems*. [S.l.]: Morgan Kaufmann, 1990. v. 2, p. 598–605. Citado na página 20.

LI, H. et al. Pruning filters for efficient convnets. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 11 vezes nas páginas 19, 23, 29, 31, 33, 43, 44, 45, 49, 60 e 61.

LIU, Z. et al. Rethinking the value of network pruning. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado na página 23.

LOSHCHILOV, I.; HUTTER, F. Decoupled weight decay regularization. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado na página 52.

LUCCIONI, A. S.; VIGUIER, S.; LIGOZAT, A.-L. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *arXiv preprint arXiv:2211.02001*, 2022. Citado na página 15.

MERITY, S. et al. Pointer sentinel mixture models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 3 vezes nas páginas 35, 38 e 39.

MICHEL, P.; LEVY, O.; NEUBIG, G. Are sixteen heads really better than one? In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2019. v. 32. Citado 5 vezes nas páginas 13, 21, 43, 48 e 61.

MICIKEVICIUS, P. et al. Mixed precision training. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2018. Citado na página 52.

NAIR, V.; HINTON, G. E. Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2010. Citado 2 vezes nas páginas 5 e 14.

NIELSEN, M. A. *Neural Networks and Deep Learning*. [S.l.]: Determination Press, 2015. Disponível em: <http://neuralnetworksanddeeplearning.com/>. Citado 3 vezes nas páginas 3, 4 e 5.

NVIDIA Corporation. *NVIDIA T4 Tensor Core GPU: Datasheet*. 2019. Disponível em: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf>. Acesso em: 28 set. 2026. Citado 2 vezes nas páginas 25 e 52.

NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. 2020. White paper. Descreve o suporte a esparsidade estruturada 2:4 na arquitetura Ampere. Citado na página 19.

PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 35 e 53.

PATTERSON, D. et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021. Citado na página 15.

RADFORD, A. et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019. Citado 8 vezes nas páginas 14, 25, 35, 36, 37, 38, 39 e 58.

RAJBHANDARI, S. et al. ZeRO: Memory optimizations toward training trillion parameter models. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. [S.l.]: IEEE, 2020. p. 1–16. Citado 3 vezes nas páginas 25, 36 e 37.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 7.

SANH, V. et al. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. Citado 2 vezes nas páginas 36 e 37.

SANH, V.; WOLF, T.; RUSH, A. M. Movement pruning: Adaptive sparsity by fine-tuning. In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2020. v. 33, p. 20378–20389. Citado na página 43.

SCHWARTZ, R. et al. Green AI. *Communications of the ACM*, v. 63, n. 12, p. 54–63, 2020. Citado na página 15.

SEVILLA, J. et al. Compute trends across three eras of machine learning. *arXiv preprint arXiv:2202.05924*, 2022. Citado na página 16.

SNOEK, J.; LAROCHELLE, H.; ADAMS, R. P. Practical Bayesian optimization of machine learning algorithms. In: *Advances in Neural Information Processing Systems (NIPS)*. [S.l.: s.n.], 2012. v. 25, p. 2951–2959. Citado na página 8.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in nlp. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 15 e 25.

SUN, M. et al. A simple and effective pruning approach for large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2024. Citado 3 vezes nas páginas 36, 37 e 43.

TOUVRON, H. et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. Citado 2 vezes nas páginas 25 e 36.

VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2017. p. 5998–6008. Citado 3 vezes nas páginas 10, 12 e 13.

WOLF, T. et al. Transformers: State-of-the-art natural language processing. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. [S.l.]: Association for Computational Linguistics, 2020. p. 38–45. Citado 2 vezes nas páginas 26 e 53.

YANG, T.-J.; CHEN, Y.-H.; SZE, V. Designing energy-efficient convolutional neural networks using energy-aware pruning. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2017. Citado na página 23.

ZHANG, S. et al. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022. Citado 3 vezes nas páginas 25, 36 e 37.

ZHU, M.; GUPTA, S. To prune, or not to prune: Exploring the efficacy of pruning for model compression. *arXiv preprint arXiv:1710.01878*, 2017. Citado na página 22.

8 Apêndice

[TO DO]